MRSpinDynamics

Python User Manual

Python magnetic-resonance spin-dynamics workspace

Simulation workflows for magnetic-resonance spin dynamics in possibly non-uniform and time-varying fields, with validated MATLAB-compatible spin-1/2 NMR Bloch kernels, Python-native analysis tools, scoped scalar-coupled spin-1/2 models, pulsed NQR extensions, and first ESR/EPR Hamiltonian and pulse-simulation helpers.

June 2026

Contents

1 Physical Scope and Assumptions

The MATLAB-compatible workflow layer in PythonSpinDynamics models a bath of uncoupled spin-1/2 nuclei evolving in a static, spatially non-uniform, or time-varying main field $B_0(\mathbf{r}, t)$ and applied RF fields. The elementary object in that layer is an isochromat: a subensemble of spin-1/2 nuclei that shares an offset, RF sensitivity, relaxation constants, and probe response.

Modeled directly in the core Bloch workflows. Independent spin-1/2 magnetization vectors; offset distributions from $B_0(\mathbf{r}, t)$; RF rotations; free precession; phenomenological T_1 and T_2 relaxation; probe transmit and receive filtering; optional deterministic radiation damping; optional motion through field maps.

Scalar-coupling extension. The `spin_dynamics.coupling` namespace adds small dense spin-1/2 systems with scalar J couplings, analytic low-field J-editing models, ideal TANGO-B filter curves, and initial SLIC simulations. These models are intentionally scoped and are documented in Chapter 5.

Pulsed NQR extension. The `spin_dynamics.nqr` namespace adds dense single-site quadrupolar spin tools for selective pulsed NQR, including spin-1 and spin-3/2 transition metadata, single-crystal and powder orientation averages, SLSE echo trains, two-frequency population transfer, EFG inhomogeneity, finite-window SLSE spectra, and weak-B0 perturbations. These models are documented in Chapter 7.

ESR/EPR extension. The `spin_dynamics.esr` namespace adds single-electron spin-1/2 ESR tools, including scalar or anisotropic g tensors, single-crystal and powder spectra, CW absorption/derivative lineshapes, static g-strain and field-strain distributions, rectangular pulsed ESR FID and Hahn-echo simulations with Liouville-space T_1/T_2 relaxation, and a first dense isotropic electron-nuclear hyperfine model. These tools are documented in Chapter 6.

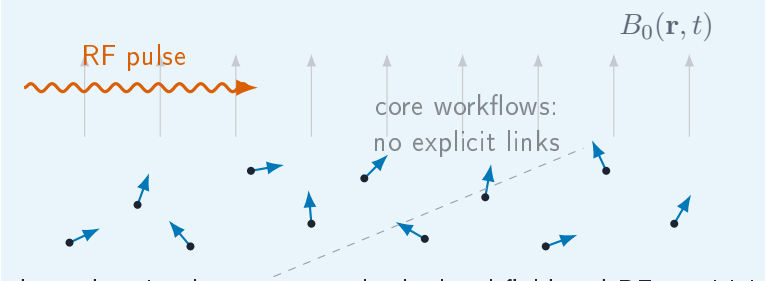
Chemical / site exchange extension. The `spin_dynamics.exchange` namespace adds Bloch-McConnell site exchange: a bath of magnetically distinct sites that swap magnetization at finite first-order kinetic rates while each site relaxes with its own T_1/T_2 and precesses at its own offset. It provides kinetic generators with detailed-balance checks, transverse lineshape coalescence, longitudinal mixing propagators, and encode-mix-detect T_2-T_2 relaxation exchange (REXSY) data that inverts to an exchange map. These tools are documented in Chapter 8.

Internal / susceptibility gradients. The `spin_dynamics.susceptibility` namespace generates the static internal field produced by magnetic-susceptibility contrast in porous media. It provides analytic two-dimensional cylindrical-grain off-resonance maps that feed the moving-isochromat pipeline and pore-space internal-gradient distributions for diffusion-in-internal-gradient studies. These tools are documented in Chapter 8.

Still outside the package scope. Dipolar coupling networks, arbitrary nonselective multi-quantum pulse sequences, shaped sample microstructure beyond the supplied field and density maps, microscopic spin noise, and large sparse coupled-spin simulations. Spin-spin effects enter the Bloch workflows only indirectly when represented by effective relaxation times such as T_2 or T_2^*. Site exchange is modeled phenomenologically by the `spin_dynamics.exchange` layer rather than from a microscopic exchange Hamiltonian.

This scope is deliberate. The main package follows MATLAB's classical Bloch/isochromat viewpoint, which is appropriate for many pulse, probe, imaging, and relaxation studies where the spins can be treated as independent subensembles. The coupled-spin namespace adds targeted quantum spin-1/2 tools for scalar J-coupling phenomena without turning the full package into a general quantum spin Hamiltonian framework.

1.1 Spin Bath Cartoon



Independent isochromats sample the local field and RF sensitivity.

1.2 Consequences for Users

- Use offset grids, field maps, or moving particles to represent spatially varying B_0, transmit B_1, and receive B_1 effects.
- Use relaxation constants to represent ensemble dephasing and energy exchange that are outside the explicit isochromat model.
- Use `spin_dynamics.coupling` when a small spin-1/2 scalar J-coupled model is the phenomenon of interest.
- Use `spin_dynamics.esr` for single-electron ESR spectra, pulsed ESR FID/echo simulations, static disorder studies, and simple isotropic electron-nuclear hyperfine doublets.
- Use `spin_dynamics.nqr` when selective pulsed NQR of a small quadrupolar site is the phenomenon of interest.
- Do not use the package for chemical exchange, large coupled-spin networks, or arbitrary nonselective multi-quantum coherence pathways.

2 Overview

PythonSpinDynamics is a Python port of the MATLAB spin-dynamics simulation package. The MATLAB tree under `../MATLABSpinDynamics/SpinDynamicsUpdated/Version_2/code` remains the reference for numerical behavior, while this workspace provides a typed, testable Python API for simulations, validation fixtures, examples, and new analysis workflows.

The supported surface currently includes ideal, tuned, untuned, and matched probe CPMG workflows; Uhrig dynamical decoupling (UDD) timing and moving-isochromat sequence helpers; finite echo trains; FID; phase-encoded and frequency-encoded (spin-warp / RARE) imaging; diffusion CPMG; time-varying-field CPMG; WURST inversion and matched WURST-CPMG; radiation damping; moving-isochromat motion helpers; received-signal noise; inverse-Laplace analysis; compact pulse-optimization scaffolds; and small spin-1/2 scalar-coupling tools for low-field J-editing, TANGO-B filtering, and SLIC-style spin-lock response; and early pulsed NQR helpers for quadrupolar sites, powder averaging, SLSE, two-frequency population transfer, EFG inhomogeneity, spin-3/2 transition metadata, and weak-static-field spectra.

2.1 Package Layout

Path	Purpose
`src/spin_dynamics/`	Installable Python package.
`examples/`	CLI examples and plotting scripts.
`tests/`	Smoke tests, validation tests, and unit tests.
`validation/fixtures/`	MATLAB/Octave fixture arrays.
`validation/octave/`	Fixture generation scripts.
`docs/python_api/`	Lightweight Markdown API notes.
`docs/validation_results.md`	Current validation status and tolerance notes.
`docs/radiation_damping.md`	Focused radiation-damping model notes.

3 Installation and Test Runs

3.1 Editable Install

PythonSpinDynamics requires Python 3.10 or newer. The core install depends on NumPy 1.24 or newer and does not require MATLAB at runtime. MATLAB is only needed to regenerate the complete MATLAB reference fixture set; Octave can regenerate the smaller dependency-light fixtures.

From `PythonSpinDynamics`, install the package into an active environment:

```
python -m pip install -e .
```

Optional extras are provided for development, plotting, benchmarking, and SciPy-backed optimization:

```
python -m pip install -e ".[dev,opt,plot,bench]"
```

The core package depends only on NumPy. The `opt` extra enables SciPy-backed continuous optimizers, non-negative inverse-Laplace solves, and MATLAB `.mat` result import/export helpers. The `plot` extra enables the Matplotlib examples.

3.2 Tests

Use the smoke tier during normal editing:

```
python -m unittest tests.smoke_tests
```

Run the full validation suite before committing numerical or workflow changes:

```
python -m unittest discover -s tests
```

4 Model Conventions

4.1 Coherence Ordering

Low-level kernels preserve MATLAB's coherence ordering:

$$\mathbf{M} = \begin{bmatrix} M_0 & M_- & M_+ \end{bmatrix}^{\mathsf{T}}.$$

This ordering differs from many Bloch-vector presentations, so new kernels and validation tests should state the expected ordering explicitly.

4.2 Normalized Time

The core MATLAB routines usually normalize time to the nominal RF amplitude $\omega_1 = 1$. In these units, a 90-degree hard pulse has duration

$$t_{90} = \frac{\pi}{2},$$

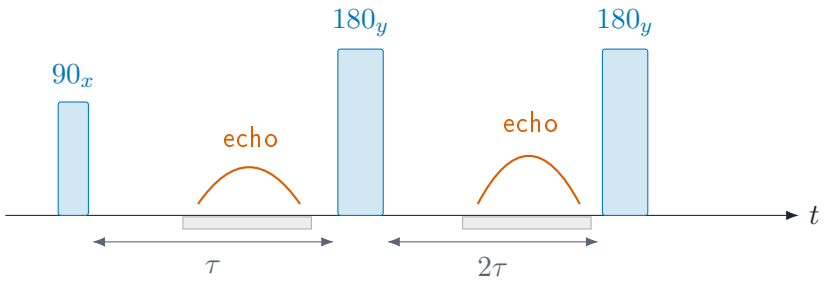
and a 180-degree hard pulse has duration

$$t_{180} = \pi.$$

Workflow helpers that expose physical seconds convert into these normalized units before calling the lower-level kernels.

4.3 Pulse Timing Cartoon

A minimal spin-echo or CPMG building block is a sequence of RF pulses, free precession windows, and acquisition windows. The package represents these as ordered intervals with pulse matrices and free-precession delays.



4.4 Effective Rotations

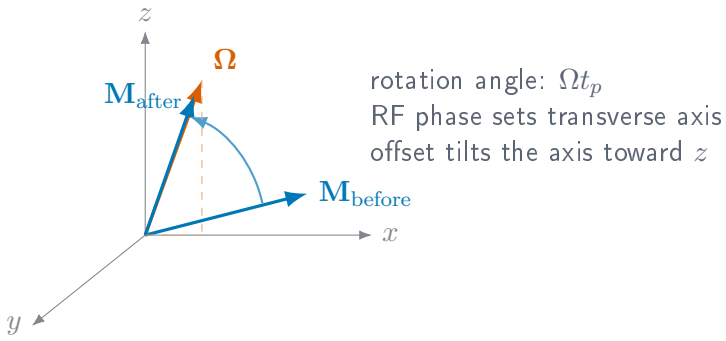
For an isochromat with normalized offset $\Delta\omega$, RF amplitude ω_1, phase ϕ, and pulse duration t_p, the rotating-frame effective field is

$$\boldsymbol{\Omega} = \begin{bmatrix} \omega_1 \cos\phi \\ \omega_1 \sin\phi \\ \Delta\omega \end{bmatrix}, \qquad \Omega = \sqrt{\omega_1^2 + \Delta\omega^2}.$$

The magnetization update is a rotation by angle Ωt_p about $\boldsymbol{\Omega}/\Omega$. The Python rotation helpers expose this calculation through `spin_dynamics.core.rotations` and return array shapes chosen to match the MATLAB reference fixtures.

4.5 Rotation Cartoon

In the rotating frame, a rectangular RF segment rotates the magnetization about an effective axis set by RF phase, RF amplitude, and off-resonance offset.



4.6 Relaxation

Free-precession intervals with relaxation use the usual exponential factors:

$$M_{xy}(t+\Delta t) = M_{xy}(t)\exp\left(-\frac{\Delta t}{T_2}\right)\exp(-\mathrm{i}\Delta\omega\,\Delta t),$$

$$M_z(t+\Delta t) = M_{\mathrm{th}} + (M_z(t) - M_{\mathrm{th}})\exp\left(-\frac{\Delta t}{T_1}\right).$$

Finite acquisition workflows assemble these intervals from pulse-sequence parameters and then call the `arb1D`-style kernels.

4.7 Echo Conversion

The echo utilities direct-sum a received spectrum $S(\Delta\omega)$ over the offset grid:

$$e(t) = \sum_k S(\Delta\omega_k)\exp(\mathrm{i}\Delta\omega_k t)\,\Delta\omega.$$

The implementation uses the same direct-sum convention as the MATLAB `calc_time_domain_echo` reference path. FID traces use the corresponding FID conversion helper.

4.8 Radiation Damping

The deterministic radiation-damping back-action model follows the local Measurements text convention:

$$T_{\mathrm{rd}} = \frac{2}{\gamma\mu_0\eta M_0 Q},$$

where γ is the gyromagnetic ratio, η is the magnetic-energy fill factor, M_0 is the equilibrium magnetization density, and Q is the loaded probe quality factor.

The analytic on-resonance hard-pulse FID envelope used by the test suite is

$$|M_{xy}(t)| = \frac{M_0}{\cosh\left(t/T_{\mathrm{rd}} - \log\tan(\theta/2)\right)}.$$

The finite-sequence implementation can add feedback during free-precession and acquisition windows, and can optionally apply an operator-split approximation during RF pulse matrices.

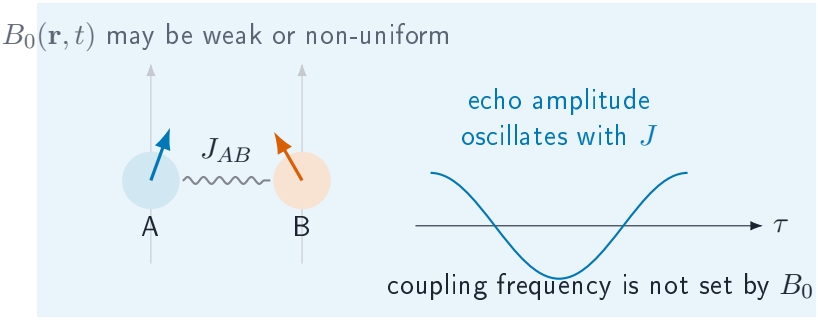
5 Scalar J-Coupling Models

Scalar J coupling is field independent, so it can remain visible in weak or grossly inhomogeneous magnetic fields where chemical-shift resolution is compressed. The `spin_dynamics.coupling` namespace adds a scoped set of spin-1/2 tools for these cases. It is separate from the MATLAB-compatible Bloch workflow layer: Bloch workflows propagate independent isochromats, while the coupling layer propagates small dense Hilbert-space systems or evaluates closed-form J-editing response curves.

Use this layer for: heteronuclear low-field J-editing curves, known-J spectrum fitting, ideal TANGO-B filter selectivity, and exploratory two-spin or few-spin scalar-coupled Hamiltonian simulations.

Current limits: spin-1/2 only; dense matrices only; no relaxation superoperators; no chemical exchange; no quadrupolar nuclei; no large sparse coupled-spin networks; no full pulse-sequence compiler for arbitrary multi-quantum pathways.

5.1 J-Coupled Spin Cartoon



5.2 Dense Spin-1/2 Hamiltonians

A coupled system is built from offsets ν_i in hertz and symmetric scalar couplings J_{ij} in hertz:

```
from spin_dynamics.coupling import (
    coupled_spin_system,
    isotropic_j_hamiltonian,
    zeeman_hamiltonian,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
    labels=["A", "B"],
)
hamiltonian = zeeman_hamiltonian(system) + isotropic_j_hamiltonian(system)
```

Hamiltonian builders return angular-frequency matrices in radians per second and use spin-1/2 operators

with eigenvalues $\pm 1/2$. The rotating-frame offset Hamiltonian is

$$H_Z = 2\pi \sum_i \nu_i I_{iz}.$$

For weakly coupled systems, the secular scalar Hamiltonian is

$$H_J^{\mathrm{sec}} = 2\pi \sum_{i<j} J_{ij} I_{iz} I_{jz},$$

while the isotropic scalar Hamiltonian is

$$H_J^{\mathrm{iso}} = 2\pi \sum_{i<j} J_{ij} \left(I_{ix} I_{jx} + I_{iy} I_{jy} + I_{iz} I_{jz} \right).$$

The dense propagator evaluates

$$\rho(t) = U(t)\rho(0)U^{\dagger}(t), \qquad U(t) = \exp\left(-\mathrm{i}Ht\right),$$

and is intended for small systems where dense matrices are acceptable.

5.3 Inhomogeneous B0/B1 Isochromats

The bridge between the coupled-spin layer and the Bloch workflow layer is a coupled isochromat ensemble. Each isochromat contains the same scalar-coupled spin network, but samples a different local field and receive weight. For isochromat k, the local spin offsets are

$$\nu_{i,k} = \nu_i^{(0)} + s_i \, \Delta\nu_k,$$

where $\nu_i^{(0)}$ is the base spin offset, $\Delta\nu_k$ is the local B0 offset in hertz, and s_i is a per-spin offset scale. The default $s_i = 1$ adds the same field offset to every spin. Heteronuclear or carrier-specific models can supply different scales.

During an RF or spin-lock interval, the nominal nutation frequency is scaled by the local transmit field:

$$\omega_{1,k} = b_{1,k}^{\mathrm{tx}} \omega_1.$$

The ensemble signal is accumulated as

$$S = \sum_k w_k b_{1,k}^{\mathrm{rx}} \, \mathrm{Tr}(\rho_k D),$$

where w_k is the isochromat weight, $b_{1,k}^{\mathrm{rx}}$ is the local receive scale, and D is the detection operator.

```
from spin_dynamics.coupling import (
    coupled_isochromat_ensemble,
    free_precession_step,
    rf_step,
    simulate_coupled_isochromat_sequence,
)

ensemble = coupled_isochromat_ensemble(
    system,
    b0_offsets_hz=[-2.0, 0.0, 2.0],
```

```
    weights=[0.25, 0.5, 0.25],
    b1_tx_scale=[0.9, 1.0, 1.1],
    b1_rx_scale=[1.0, 1.0, 0.8],
)

result = simulate_coupled_isochromat_sequence(
    ensemble,
    [
        rf_step(duration=0.25, nutation_hz=1.0, phase=1.5708),
        free_precession_step(duration=0.01),
    ],
    initial_axis="x",
    detect_axis="x",
)
```

For time-varying fields, each sequence step may override the ensemble B0 offsets or B1 transmit scale. This keeps the first implementation close to the existing isochromat approach: the coupled-spin Hilbert space is dense and small, while field inhomogeneity is represented by an outer ensemble sum.

5.4 Low-Field J-Editing

Analytic J-editing helpers model response curves as sums of field-independent cosine modulations:

$$s(\tau) = b + \sum_m a_m \cos^{p_m}(2\pi N J_m \tau)\,,$$

where τ is the encoding time, N is the number of encoding cycles, J_m is a known or hypothesized coupling in hertz, a_m is its amplitude, and p_m is an optional multiplicity exponent. Proton-detected models use $p_m = 1$. Carbon-detected CH_n groups use $p_m = n$.

```
from spin_dynamics.coupling import (
    carbon_detected_j_modulation,
    fit_known_j_spectrum,
    j_modulation_curve,
)

signal = j_modulation_curve(
    encoding_times,
    couplings_hz=[125.0, 160.0],
    amplitudes=[0.85, 0.15],
)

carbon_signal = carbon_detected_j_modulation(
    encoding_times,
    couplings_hz=[125.0, 160.0],
    abundances=[0.85, 0.15],
    proton_counts=[2, 1],
)

fit = fit_known_j_spectrum(
    encoding_times,
    signal,
    couplings_hz=[125.0, 160.0],
    include_background=False,
```

```
)
```

`fit_known_j_spectrum` solves a linear least-squares problem once the candidate J_m values are supplied. It returns amplitudes, optional background, fitted signal, residual, rank, and residual norm.

5.5 TANGO-B Filtering

The ideal TANGO-B helper evaluates the coupled-spin transverse filter envelope

$$A(J; t_d) = \sin^2(\pi J t_d)\,.$$

When a target coupling $J_\star$ and positive odd order n are supplied, the delay is selected as

$$t_d = \frac{n}{2J_\star}.$$

```
from spin_dynamics.coupling import tango_b_filter

response = tango_b_filter(
    couplings_hz,
    target_coupling_hz=160.0,
    order=3,
)
```

This model is idealized: it reports the scalar-coupling selectivity envelope, not RF imperfections, relaxation, diffusion, or probe filtering.

5.6 SLIC Response

Spin-lock induced crossing (SLIC) is modeled by adding an RF spin-lock Hamiltonian to the static coupled-spin Hamiltonian:

$$H_{\mathrm{SLIC}} = H_Z + H_J^{\mathrm{iso}} + 2\pi\omega_1 \sum_i I_{ix}.$$

The helper scans spin-lock nutation frequencies ω_1 in hertz and returns the remaining transverse magnetization and its corresponding dip:

```
from spin_dynamics.coupling import (
    coupled_spin_system,
    simulate_slic_spectrum,
    two_spin_slic_transfer_time,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
)
duration = two_spin_slic_transfer_time(offset_difference_hz=0.7)
result = simulate_slic_spectrum(
    system,
    nutation_frequencies_hz,
    spin_lock_time=duration,
)
```

For a nearly equivalent two-spin system, the SLIC dip is expected near the coupling frequency. The helper is exploratory and dense; broad homonuclear networks will need sparse or specialized methods.

6 Electron Spin Resonance Models

The `spin_dynamics.esr` namespace adds an ESR/EPR extension for single-electron spin-1/2 systems. The first layer is intentionally compact: it uses dense matrices, explicit g-tensor axes, and small product Hilbert spaces for hyperfine examples. Hamiltonians are expressed in radians per second, while public resonance frequencies, linewidths, and hyperfine constants are expressed in hertz.

Use this layer for: single-electron ESR resonance calculations, anisotropic g-tensor single-crystal and powder spectra, CW absorption or first-derivative display, static g-strain and field-strain broadening, rectangular pulsed ESR FID and Hahn-echo simulations, Liouville-space T_1/T_2 relaxation, and first electron-nuclear isotropic hyperfine doublets.

Current limits: electron spin $S = 1/2$ only in the main ESR system; dense matrices only; hyperfine couplings are isotropic $A\,\mathbf{S}\cdot\mathbf{I}$; no anisotropic A tensors, exchange, electron-electron dipolar coupling, higher-spin zero-field splitting, saturation, or microwave resonator model yet.

6.1 Zeeman Hamiltonian and g Tensors

The single-electron Zeeman Hamiltonian is

$$H_Z = 2\pi \frac{\mu_B}{h}\,\mathbf{B}_0^{\mathsf{T}}\,\mathbf{g}\,\mathbf{S},$$

where μ_B/h is exposed as `BOHR_MAGNETON_HZ_PER_T`. The g tensor may be supplied as a scalar, a three-component principal-axis vector, or a full 3×3 matrix. For a static-field direction $\hat{\mathbf{n}}$, the effective g value is

$$g_{\mathrm{eff}}(\hat{\mathbf{n}}) = \left|\mathbf{g}^{\mathsf{T}}\hat{\mathbf{n}}\right|,$$

and the spin-1/2 transition frequency is

$$\nu_{\mathrm{ESR}} = \frac{\mu_B}{h}\,B_0\,g_{\mathrm{eff}}.$$

```
from spin_dynamics.esr import (
    ESRSpinSystem,
    effective_g_value,
    resonance_field_tesla,
    resonance_frequency_hz,
)

system = ESRSpinSystem(g_tensor=[2.00, 2.08, 2.24])
g_z = effective_g_value(system, [0.0, 0.0, 1.0])
nu = resonance_frequency_hz(system, [0.0, 0.0, 0.34])
b_res = resonance_field_tesla(system, microwave_frequency_hz=9.5e9)
```

6.2 CW Spectra, Lineshapes, and Disorder

Frequency-swept and field-swept helpers broaden orientation-resolved ESR lines with Gaussian or Lorentzian profiles. The same helpers can return absorption or first-derivative CW-style spectra:

```
from spin_dynamics.esr import simulate_field_sweep

result = simulate_field_sweep(
    system,
    microwave_frequency_hz=9.5e9,
    orientations="powder",
    broadening_tesla=0.2e-3,
    lineshape="lorentzian",
    detection_mode="derivative",
)
```

Static disorder is represented by weighted samples. The convenience `static_disorder_grid` builds Gaussian quadrature-like samples for diagonal g-strain and applied-field offsets:

```
from spin_dynamics.esr import (
    simulate_field_sweep_distribution,
    static_disorder_grid,
)

samples = static_disorder_grid(
    system,
    g_std=[0.0, 0.0, 0.005],
    field_std_tesla=0.1e-3,
    g_points=5,
    field_points=5,
)

strained = simulate_field_sweep_distribution(
    samples,
    microwave_frequency_hz=9.5e9,
    orientations="powder",
    broadening_tesla=0.05e-3,
)
```

6.3 Pulsed ESR

The pulsed ESR helpers use a single rotating frame and a rotating-wave approximation. The `nutation_hz` parameter is the on-resonance spin-1/2 Rabi rate for the selected microwave B_1 direction, so a rectangular pulse of flip angle θ has duration

$$t_p = \frac{\theta}{2\pi\nu_1}.$$

In particular, a 90-degree pulse has duration $1/(4\nu_1)$.

```
import numpy as np

from spin_dynamics.esr import (
    flip_angle_duration,
    simulate_fid,
    simulate_hahn_echo,
)

carrier = resonance_frequency_hz(system, [0.0, 0.0, 0.34])
nu1 = 5e6
```

```
t90 = flip_angle_duration(np.pi / 2.0, nu1)
t180 = flip_angle_duration(np.pi, nu1)

fid = simulate_fid(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    pulse_duration_seconds=t90,
    times_seconds=np.linspace(0.0, 5e-6, 256),
    rf_frequency_hz=carrier,
)

echo = simulate_hahn_echo(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    excitation_duration_seconds=t90,
    refocus_duration_seconds=t180,
    tau_seconds=2e-6,
    times_seconds=np.linspace(0.0, 4e-6, 256),
    rf_frequency_hz=carrier,
)
```

6.4 Pulsed Relaxation

For physical T_1/T_2 relaxation during pulses, free evolution, and acquisition, pass an `ESRRelaxationModel`. The model acts in Liouville space on the trace-zero high-temperature density-matrix deviation: T_1 damps population differences while preserving trace, and T_2 damps coherences.

```
from spin_dynamics.esr import ESRRelaxationModel

relaxed = simulate_fid(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    pulse_duration_seconds=t90,
    times_seconds=np.linspace(0.0, 5e-6, 256),
    rf_frequency_hz=carrier,
    relaxation=ESRRelaxationModel(t1_seconds=50e-6, t2_seconds=3e-6),
)
```

The older `t2_seconds` argument in the pulsed helpers is a simple post-propagation envelope retained for quick examples. When using `ESRRelaxationModel`, leave `t2_seconds=inf` to avoid double-counting coherence decay.

6.5 Isotropic Hyperfine Coupling

The first hyperfine layer models one electron spin-1/2 coupled isotropically to one or more nuclei:

$$H = H_{eZ} + H_{nZ} + 2\pi \sum_k A_k \, \mathbf{S} \cdot \mathbf{I}_k,$$

with A_k in hertz. The helper scans the applied field, diagonalizes the full dense electron-nuclear Hamiltonian at each point, and sums ESR-active transitions according to the microwave B_1 matrix element.

```
from spin_dynamics.esr import (
    NuclearSite,
    electron_nuclear_system,
    simulate_hyperfine_field_sweep,
)

hf_system = electron_nuclear_system(
    [20e6],
    nuclei=[NuclearSite("H1", isotope="1H", gamma_hz_per_t=42.577e6)],
    g_tensor=2.0023,
)

hf_sweep = simulate_hyperfine_field_sweep(
    hf_system,
    microwave_frequency_hz=9.5e9,
    broadening_hz=0.5e6,
)
```

For one spin-1/2 nucleus in the high-field limit, the ESR line splits by approximately A in frequency, or by

$$\Delta B \approx \frac{A}{(\mu_B/h)g}$$

in a field-swept spectrum.

7 Pulsed NQR Models

Nuclear quadrupole resonance is driven by the interaction between a nuclear quadrupole moment and the local electric-field-gradient (EFG) tensor. The `spin_dynamics.nqr` namespace adds a scoped quadrupolar extension for solid-state pulsed NQR experiments. It is intentionally separate from the spin-1/2 Bloch and scalar-coupling layers.

Use this layer for: reduced two-level (selective) pulsed NQR of spin-1 sites; full $(2I+1)$-level density-matrix FID and echo dynamics required for spin-3/2; automatic reduced-vs-full model selection; spin-1 and spin-3/2 transition metadata; single-crystal and powder averages over local EFG orientations; SLSE echo trains; two-frequency population transfer; static EFG inhomogeneity; finite-window SLSE spectra; and weak-B0 line splitting.

Current limits: dense single-site matrices only; two pulse models (the reduced spin-1 selective pulse and the full $(2I+1)$-level density-matrix pulse). The full-model pulse uses a single-carrier rotating-wave approximation addressing one transition band, so it covers spin-3/2 zero-field and weak-Zeeman dynamics but is not yet a general multi-band higher-spin solver. The reduced SLSE and population-transfer workflows remain spin-1; relaxation is phenomenological rather than microscopic Redfield/dipolar; no multi-site coupling between quadrupolar nuclei.

7.1 Quadrupolar Site Model

A quadrupolar site is described in the EFG principal-axis system. The zero-field Hamiltonian used by the Python helper is

$$H_Q = 2\pi\nu_{\mathrm{scale}}\left[3I_z^2 - I(I+1)\mathbf{1} + \eta\left(I_x^2 - I_y^2\right)\right],$$

where I is the spin quantum number, ν_Q is the quadrupole-frequency parameter in hertz, and $0 \le \eta \le 1$ is the EFG asymmetry parameter. Hamiltonians are represented in radians per second. For spin-1, the package uses $\nu_{\mathrm{scale}} = \nu_Q/3$, matching the nitrogen-14 convention where the zero-field lines are $\nu_x = \nu_Q(1+\eta/3)$, $\nu_y = \nu_Q(1-\eta/3)$, and $\nu_z = 2\eta\nu_Q/3$. For spin-3/2, $\nu_{\mathrm{scale}} = \nu_Q/6$, so the zero-field chlorine-style NQR line is

$$\nu_{\mathrm{NQR}} = \nu_Q\sqrt{1+\eta^2/3}.$$

Zero-frequency Kramers-doublet transitions are omitted from the transition inventory.

Weak Zeeman perturbations can be added as

$$H_Z = -2\pi\gamma\,\mathbf{B}_0 \cdot \mathbf{I},$$

where $\mathbf{B}_0$ is expressed in the EFG principal-axis system for the current crystallite orientation. The default is $B_0 = 0$, which is the usual NQR case.

```
from spin_dynamics.nqr import QuadrupolarSite, diagonalize_site

site = QuadrupolarSite(
    spin=1,
    isotope="14N",
    quadrupole_frequency_hz=900e3,
    eta=0.3,
```

```
)

eigensystem = diagonalize_site(site)
for transition in eigensystem.transitions:
    print(transition.label, transition.frequency_hz)
```

For spin-1 at zero field, transitions are labeled by their dominant RF polarization in the EFG principal-axis system: `x`, `y`, and `z`. For the example above the line positions are approximately 990, 810, and 180 kHz.

Spin-3/2 transition metadata can be inspected with the same diagonalization helper:

```
chlorine = QuadrupolarSite(
    spin=1.5,
    isotope="35Cl",
    quadrupole_frequency_hz=30e6,
    eta=0.1,
    gamma_hz_per_t=4.171e6,
)

for transition in diagonalize_site(chlorine).transitions:
    print(transition.frequency_hz)
```

7.2 Two Modeling Regimes

Before propagating a pulse, the package chooses between two models. The choice is governed by how many states the RF pulse can connect, not by the number of spectral lines. The detailed reasoning is given in the technical note `References/Pulsed_NQR_Spin_Dynamics_Narrative_Rewrite`.

- **Reduced two-level model** (`SelectivePulse`, `simulate_slse`): one pulse addresses one transition as an embedded fictitious spin-1/2 rotation. This is honest only when the selected transition is isolated. For a target transition t, with $\Delta_{\text{iso}} = \min_{j \neq t} |\omega_t - \omega_j|$ the spacing to the nearest RF-active competitor, the validity condition is

$$\Delta_{\text{iso}} \gg \max(\Omega_1,\ \Delta\omega_{\text{pulse}},\ \Gamma)\,,$$

  where Ω_1 is the effective nutation rate, $\Delta\omega_{\text{pulse}} \sim 1/t_p$ is the pulse bandwidth, and Γ is the linewidth. A nonzero η is necessary but *not* sufficient: at $\eta = 0$ the spin-1 x and y lines coincide and the reduction fails. The reduced path is restricted to spin-1 for this reason.

- **Full density-matrix model** (`spin_dynamics.nqr.full_dynamics`): the complete $(2I+1)$-level density matrix is propagated. This is the correct general model and is required for spin-3/2, whose single zero-field line connects two Kramers doublets—four states hidden behind one frequency.

7.3 Selective Pulse Approximation

Most practical spin-1 pulsed NQR RF pulses are narrowband relative to the separation between quadrupolar transitions. The reduced model uses this selective limit: one pulse addresses one transition and acts as an embedded two-level rotation inside the full $(2I+1)$-level Hilbert space.

For a transition $|a\rangle \leftrightarrow |b\rangle$, the transition dipole vector is

$$\mathbf{d}_{ab} = (\langle a|I_x|b\rangle, \langle a|I_y|b\rangle, \langle a|I_z|b\rangle)\,.$$

For a local RF direction $\hat{\mathbf{b}}_1$, the complex drive projection is proportional to $\hat{\mathbf{b}}_1 \cdot \mathbf{d}_{ab}$. Preserving this signed complex projection is essential for powder averages because transmit and receive phases must pair consistently across crystallites.

```
from spin_dynamics.nqr import SelectivePulse, apply_selective_pulse

pulse = SelectivePulse(
    "x",
    duration_seconds=50e-6,
    nutation_hz=10e3,
)
```

The pulse helper also supports an optional RF frequency. If omitted, the RF is placed on the selected transition. If supplied, the difference between the RF frequency and transition frequency is treated as a two-level detuning.

The selective-pulse propagation routines support spin-1 sites only. Spin-3/2 nuclei such as $^{35}\mathrm{Cl}$ and $^{37}\mathrm{Cl}$ have degenerate doublets at zero field, so their pulsed response is handled by the full density-matrix model below rather than by an embedded two-level transition; calling the reduced selective-pulse routines on a non-spin-1 site raises a clear error.

7.4 Full Density-Matrix Model

The `spin_dynamics.nqr.full_dynamics` module propagates the complete $(2I+1)$-level density matrix in a rotating frame at the pulse carrier. It is the required model for spin-3/2 and also runs for spin-1. The static Hamiltonian $H_Q + H_Z$ is diagonalized for the actual EFG and field orientation, the RF and detection operators $\hat{\mathbf{e}} \cdot \mathbf{I}$ are transformed into that eigenbasis, and the density matrix is propagated through each constant-Hamiltonian interval by $U_k = \exp(-iH_k\,\Delta t_k)$, $\rho_{k+1} = U_k \rho_k U_k^\dagger$.

Here `nutation_hz` is the bare field nutation $\gamma B_1/(2\pi)$, so the realized Rabi rate on a transition $a-b$ is $2\,\nu_1\,|\langle a|\hat{\mathbf{e}}_1 \cdot \mathbf{I}|b\rangle|$; the same pulse is a different flip angle on different transitions, which is the physical effect the full model captures. A single-carrier rotating-wave approximation is used, valid when one carrier addresses one transition band (the spin-3/2 zero-field and weak-Zeeman regime).

```
import numpy as np
from spin_dynamics.nqr import QuadrupolarSite, simulate_full_fid

site = QuadrupolarSite(spin=1.5, isotope="35Cl",
                       quadrupole_frequency_hz=30e6, eta=0.1)

fid = simulate_full_fid(
    site,
    nutation_hz=20e3,            # bare field nutation gamma*B1/(2*pi)
    pulse_duration_seconds=10e-6,
    times_seconds=np.linspace(0.0, 200e-6, 1024),
)
print(fid.signal)               # complex baseband signal
```

A two-pulse (Hahn-style) echo is available through `simulate_full_echo`, and an optional `relaxation=NQRRelaxationMo` adds Liouville-space T_1/T_2 decay. The lower-level primitives `pulse_hamiltonian`, `static_hamiltonian_rotating`, and `detection_operator` are exposed for custom sequences. The full model has been validated against an exact lab-frame propagator (the rotating-wave error scales as the Ω_1^2 Bloch-Siegert shift), and its spin-1

powder nutation curve reproduces the classic $119°$ maximum while the spin-3/2 curve peaks at a slightly smaller flip angle.

7.5 Model Selection

`select_nqr_model` reads the reduced-vs-full choice from the static Hamiltonian and the RF matrix elements for the coil polarization, not from the spin or the line count. It builds the pulse-addressed connected set (states the pulse can drive from the target pair, plus RF-driven near-degenerate partners) and the isolation ratio $\Delta_{\text{iso}}/\max(\Omega_1, 1/t_p, \Gamma)$.

```
from spin_dynamics.nqr import QuadrupolarSite, select_nqr_model

site = QuadrupolarSite(spin=1.5, quadrupole_frequency_hz=30e6, eta=0.1)
choice = select_nqr_model(
    site, "x",
    nutation_hz=5e3,
    pulse_duration_seconds=50e-6,
    b1_direction_pas=(1, 1, 1),
)
print(choice.recommended_model)   # "full" -- spin-3/2 Kramers doublets
print(choice.describe())          # diagnostic report
```

The reduced model is recommended only when the addressed set is a single, non-degenerate, RF-active pair *and* the isolation ratio clears `isolation_threshold` (default 5). The returned `NQRModelSelection` reports the target states, nearest competing RF-active transition, isolation distance and ratio, pulse bandwidth, the addressed-state set, a degeneracy flag, and human-readable reasons. It handles the regime-defining cases: spin-1 $\eta = 0$ and unresolved small-η splittings go full via the isolation ratio; spin-3/2 Kramers doublets go full because the addressed set spans four states; a strongly Zeeman-resolved spin-3/2 line can return reduced as a verified special case; and a polarization that makes neighboring lines RF-dark keeps a crowded spectrum reducible. The check is currently advisory—the reduced workflows do not yet invoke it automatically.

7.6 Orientations and Powder Averages

NQR is a solid-state experiment, so the signal depends on the orientation of the RF field relative to the local EFG principal axes. A single crystal is one fixed orientation. A powder is represented by a normalized spherical quadrature grid:

```
from spin_dynamics.nqr import powder_average_grid, single_crystal_orientation

single = single_crystal_orientation(alpha=0.0, beta=1.57079632679)
powder = powder_average_grid(n_theta=16, n_phi=32)
```

For weak-B0 calculations, each orientation may also specify the direction of B_0 in the EFG frame. If no separate B_0 direction is supplied, the current helper uses the local RF direction as the default field direction for that sample.

Weak-field spectra use a correlated powder grid for the static field and RF field directions. The helper `b0_b1_powder_average_grid` samples the orientation of B_0 in the EFG frame and the rotation angle of B_1 around B_0, preserving a chosen lab-frame angle between the two fields. For a conventional transverse coil one uses $B_1 \perp B_0$.

7.7 Weak Static B0 Spectra

Weak static magnetic fields are modeled by exact diagonalization of

$$H = H_Q + H_Z, \qquad H_Z = -2\pi\gamma\,\mathbf{B}_0 \cdot \mathbf{I}.$$

The helper is intended for the NQR regime

$$\epsilon_Z = \frac{|\gamma B_0|}{\nu_{\mathrm{ref}}} \ll 1,$$

and reports the largest perturbation ratio in the result. It can warn or raise if the requested field is no longer weak compared with the selected NQR line.

```
from spin_dynamics.nqr import simulate_weak_b0_spectrum

weak = simulate_weak_b0_spectrum(
    chlorine,
    b0_tesla=1e-3,
    orientations="powder",
    broadening_hz=200.0,
    weak_ratio_threshold=0.05,
)

print(weak.max_perturbation_ratio)
```

For each crystallite, the code diagonalizes $H_Q + H_Z$, keeps transitions near the selected zero-field NQR line, and weights them by the RF projection

$$I_{ab} \propto w_k \left|\hat{\mathbf{b}}_{1,k} \cdot \mathbf{d}_{ab,k}\right|^2,$$

where w_k is the orientation weight. This RF projection is important for spin-3/2 in weak fields: the eigenvalue splitting can contain several nearby transitions, but transitions dark to the coil should not contribute to the plotted spectrum. In the axial limit $\eta = 0$, $B_0 \parallel z_{\mathrm{PAS}}$, and $B_1 \perp B_0$, a spin-3/2 site gives the expected pair $\nu_Q \pm \gamma B_0$.

7.8 SLSE Detection

The spin-lock spin-echo (SLSE) sequence is the classic pulsed NQR detection sequence. The helper models it as repeated selective pulses on a spin-1 detection transition and reports echo amplitudes at the requested echo spacing. Echo decay can be represented by a scalar T_{2e} envelope or by a phenomenological Liouville-space relaxation model with T_1 and T_2.

```
from spin_dynamics.nqr import simulate_slse, slse_sequence

sequence = slse_sequence(
    "x",
    pulse_duration_seconds=25e-6,
    nutation_hz=10e3,
    echo_spacing_seconds=1e-3,
    num_echoes=16,
)
```

```
result = simulate_slse(
    site,
    sequence,
    orientations="powder",
    t2e_seconds=20e-3,
)
```

The returned `SLSEResult` includes echo times, powder-averaged echo amplitudes, per-orientation echo amplitudes, orientation weights, transition metadata, and the eigensystem used for the first orientation. When a relaxation model is supplied, the result also reports local cycle eigenvalues and cycle-derived effective decay times.

```
from spin_dynamics.nqr import NQRRelaxationModel

relaxed = simulate_slse(
    site,
    sequence,
    orientations="powder",
    relaxation=NQRRelaxationModel(t1_seconds=1.0, t2_seconds=20e-3),
)
```

Offset and pulse-period sweeps are available as compact diagnostics for the SLSE modulation:

```
from spin_dynamics.nqr import simulate_slse_offset_sweep

sweep = simulate_slse_offset_sweep(
    site,
    "x",
    offsets_hz=[-2e3, 0.0, 2e3],
    pulse_duration_seconds=25e-6,
    nutation_hz=10e3,
    echo_spacing_seconds=500e-6,
    relaxation=NQRRelaxationModel(t2_seconds=20e-3),
)
```

7.9 EFG Inhomogeneity and SLSE Spectra

Static EFG disorder is modeled as an isochromat ensemble of independent quadrupolar sites. Each isochromat has its own ν_Q, η, transition frequency, and normalized weight. This covers impurity-induced broadening, static temperature gradients, and other inhomogeneous mechanisms analogous to NMR isochromat broadening.

```
from spin_dynamics.nqr import (
    SelectivePulse,
    gaussian_efg_distribution,
    simulate_fid_efg_distribution,
)
import numpy as np

distribution = gaussian_efg_distribution(
    site,
    quadrupole_std_hz=2e3,
```

```
    samples=41,
)

fid = simulate_fid_efg_distribution(
    distribution,
    "x",
    times_seconds=np.linspace(0.0, 20e-3, 512),
    excitation=SelectivePulse("x", duration_seconds=2.5e-6, nutation_hz=100e3),
)
```

The distribution simulators check whether a coarse EFG grid can artificially rephase within the simulated duration. Increase the number of isochromats when the warning appears, or pass `rephase_action="ignore"` only after a convergence check.

In an SLSE experiment, the spectrum is not the FFT of the echo train. It is the FFT of the averaged echo acquired over a receiver window T_{acq} centered on a selected echo, with T_{acq} shorter than the pulse spacing. The finite acquisition window itself broadens the spectrum:

```
from spin_dynamics.nqr import simulate_slse_acquisition_spectrum

slse_spectrum = simulate_slse_acquisition_spectrum(
    distribution,
    sequence,
    acquisition_duration_seconds=200e-6,
    acquisition_points=256,
    echo_index=-1,
    noise={"target_snr": 20.0, "seed": 123},
    deconvolution_strength=1e-2,
)
```

Noise is added to the complex time-domain acquisition before the FFT. The optional deconvolution performs a regularized inversion of the same finite-window FFT operator. It is useful for exploring acquisition-window broadening, but it can amplify noise and should be checked across plausible SNR values.

7.10 Two-Frequency Population Transfer

The two-frequency NQR workflow follows the perturbation-plus-detection idea used in 2D NQR. A selective pulse on one transition changes populations in the shared three-level spin-1 manifold; a later SLSE block detects the changed amplitude on another transition. The normalized diagnostic is

$$\Delta(p,q) = \frac{S(p,q)}{S_0(q)} - 1,$$

where p is the perturbation transition and q is the detection transition.

```
from spin_dynamics.nqr import SelectivePulse, simulate_population_transfer

transfer = simulate_population_transfer(
    site,
    SelectivePulse("x", duration_seconds=50e-6, nutation_hz=10e3),
    sequence,
    orientations="powder",
)

print(transfer.normalized_difference)
```

The diagnostic plotting example constructs a compact 3×3 map over the spin-1 x, y, and z transitions. Off-diagonal features indicate that perturbation and detection frequencies are connected through the same quadrupolar site.

8 First Workflows

8.1 CPMG

Application code should normally start with the public workflow runners:

```
from spin_dynamics.workflows import run_tuned_cpmg

result = run_tuned_cpmg(numpts=101, maxoffs=10)
print(result.del_w.shape)
print(result.echo.shape)
print(result.snr)
```

The asymptotic CPMG runners return `CPMGResult` objects with the offset grid, asymptotic magnetization, received spectrum, time-domain echo, acquisition time vector, optional SNR, probe label, and the default phase-cycle metadata.

8.1.1 Uhrig Dynamical Decoupling

The sequence utility layer also provides ideal CPMG and Uhrig dynamical decoupling timing helpers. For n refocusing pulses in a total evolution window T, UDD places the jth pulse at

$$t_j = T \sin^2\left(\frac{j\pi}{2n+2}\right), \qquad j = 1, \ldots, n.$$

The pulses cluster near the beginning and end of the window. This does not usually improve simple unrestricted diffusion in a static gradient, but it can strongly suppress low-frequency correlated detuning noise.

```
from spin_dynamics.sequences import (
    cpmg_pulse_times,
    dephasing_filter_function,
    udd_pulse_times,
)

duration = 1.0
omega = 0.1 / duration
cpmg = cpmg_pulse_times(8, duration)
udd = udd_pulse_times(8, duration)

print(dephasing_filter_function(omega, cpmg, duration))
print(dephasing_filter_function(omega, udd, duration))
```

8.2 Finite Echo Trains

```
from spin_dynamics.workflows import run_matched_cpmg_train

train = run_matched_cpmg_train(
    numpts=51,
```

```
    num_echoes=8,
    auto_refine_grid=True,
    num_workers=None,
)
```

Finite train results contain one acquired spectrum and one echo per echo index. They also include trapezoidal echo integrals and physical echo-center times. The default CPMG/PAP phase-cycle table is available as `train.phase_cycle`. For long trains, use the rephasing checks or enable `auto_refine_grid=True` so the offset grid is refined before RF matrices are built.

8.2.1 CPMG-IR Trains

Finite inversion-recovery trains add an inversion delay axis before the CPMG readout:

```
from spin_dynamics.workflows import run_matched_cpmg_ir_train

ir = run_matched_cpmg_ir_train(
    numpts=31,
    num_echoes=4,
    tauvect=[0.5e-3, 2e-3, 6e-3, 12e-3],
    auto_refine_grid=True,
)
```

The returned arrays are indexed by inversion delay first and echo number second. This makes the workflow useful for compact T_1-encoded echo-train tests without manually assembling the inversion pulse, delay, excitation, and refocusing blocks.

8.2.2 Probe Parameter Sweeps

Probe-aware CPMG workflows can also be swept over resonator Q or mistuning:

```
from spin_dynamics.workflows import run_tuned_q_sweep

sweep = run_tuned_q_sweep(q_values=[20, 50, 100], numpts=51)
```

Use the finite-train sweep variants when echo-to-echo evolution matters. The finite sweep results keep one `CPMGTrainResult` per parameter value and also stack echo integrals for quick heat-map style summaries.

8.3 FID

```
from spin_dynamics.parameters import set_params_ideal_fid
from spin_dynamics.workflows.fid import sim_fid_ideal

sp, pp = set_params_ideal_fid(numpts=101)
macq, fid, tvect = sim_fid_ideal(sp, pp)
```

8.4 Imaging

```
import numpy as np
from spin_dynamics.workflows import (
    make_imaging_field_maps,
    run_tuned_phase_encoded_cpmg_imaging,
    form_imaging_image,
)

rho = np.eye(8)
fields = make_imaging_field_maps(
    rho,
    b0_map=np.zeros_like(rho),
    b1_tx_map=np.ones_like(rho),
    b1_rx_map=np.ones_like(rho),
)
result = run_tuned_phase_encoded_cpmg_imaging(
    fields,
    num_echoes=4,
    ny=9,
    receive_mode="weighted",
)
image = form_imaging_image(result, mode="echo_sum")
```

Imaging workflows return raw k-space, reconstructed per-echo images, field maps, relaxation maps, gradient metadata, and sequence timing. Image formation is intentionally separated from acquisition so selected-echo, echo-summed, fitted-ρ, and fitted-T_2 displays can be compared. `make_imaging_field_maps` accepts spatial B0, transmit-B1, and receive-B1 maps, and the ideal imaging path can include an inversion-recovery preparation through `run_t1_encoded_phase_encoded_cpmg_imaging`. When `noise` is supplied, clean k-space and image fields remain in the result while noisy counterparts are stored in `kspace_noisy`, `image_noisy`, and `magnitude_noisy`.

8.4.1 Frequency-Encoded Imaging: Spin-Warp and RARE

The phase-encoded workflows above fill k-space one point per phase-encode step. `run_spin_warp_imaging` and `run_rare_imaging` add a readout (frequency-encode) gradient during acquisition, so each spin echo samples a whole k-space line. They are built on the Lagrangian motion engine—one static isochromat per voxel—which supports the two-axis gradient waveforms that the scalar-gradient arbitrary-pulse kernels cannot express; they are ideal-probe and reuse `reconstruct_image_from_kspace`.

```
import numpy as np
from spin_dynamics.workflows import run_spin_warp_imaging, run_rare_imaging

rho = np.zeros((32, 32)); rho[8:24, 10:18] = 1.0
t2 = np.full((32, 32), 60e-3)

# Spin-warp: one spin echo per phase-encode line (pz excitations, no blurring).
ref = run_spin_warp_imaging(rho, fov=(0.02, 0.02), t2_map=t2)

# RARE / fast spin echo: each echo reads a different line, so a train of length 8
# needs only ceil(pz / 8) excitations, at the cost of T2 blurring.
fse = run_rare_imaging(rho, fov=(0.02, 0.02), t2_map=t2, echo_train_length=8)
```

Readout is along x and the phase encode along z. Each echo is gradient-balanced (an x pre-dephase and rewind return k-space to the origin before each 180-degree pulse), so the train stays a clean Meiboom-Gill

CPMG. The T2 decay across the echo train weights the phase-encode lines—`line_echo_time` records the echo time of each k_z line—which broadens the point-spread function (the RARE blurring, strongest for short T_2). `echo_train_length=1` recovers the spin-warp reference. The example `plot_rare_imaging.py` contrasts the two on a two-T_2 phantom.

Both workflows accept the same inputs as the phase-encoded path so the effects of inhomogeneity can be evaluated: pass `rho` with per-voxel `b0_map` (absolute angular off-resonance, rad/s), `b1_tx_map`, `b1_rx_map`, `t1_map`, and `t2_map`, or pass an `ImagingFieldMaps` container directly (with the map keywords omitted). B0 inhomogeneity distorts geometry along the readout axis and B1 maps shade the image. An unresolved sub-voxel B0 spread is modelled with `num_offsets` > 1 isochromats per voxel spread over $\pm$`offset_spread` (rad/s), the counterpart of the `ny`/`maxoffs` samples of the phase-encoded path; a spin echo refocuses the static spread at each echo, so it blurs the readout axis (the T_2^* point-spread function) without decaying the train. The example `plot_imaging_inhomogeneity.py` shows B0 distortion, sub-voxel T_2^* blur, and B1 shading.

A practical issue in inhomogeneous-field imaging is that the excited slice is neither flat nor uniform in real space: an RF pulse excites the spins whose frequency falls in its band, so the slice follows the curved iso-B0 contours and is shaded by B1. `imaging_slice_sensitivity` maps this sensitive slice from the same B0/B1 maps by computing, for every voxel at its own off-resonance (`b0_map - center_frequency`, rad/s) and transmit-B1, the excited transverse magnetization of a rectangular RF pulse (so the slice profile follows the pulse bandwidth $\sim 1/$`excitation_duration`), weighted by the receive B1; `refocusing=True` also applies the 180-degree refocusing efficiency for the spin-echo sensitive volume. The example `plot_sensitive_slice.py` visualizes the curved, non-uniform slice in a single-sided-like field.

8.5 Received-Signal Noise

Noise is opt-in and output-referred. High-level workflows accept a `NoiseSpec`, a mapping, or a scalar white-noise standard deviation:

```
from spin_dynamics.noise import NoiseSpec
from spin_dynamics.workflows import run_tuned_cpmg

noisy = run_tuned_cpmg(
    numpts=101,
    noise=NoiseSpec(model="probe", target_snr=25.0, seed=3),
)
```

`model="white"` adds flat complex receiver noise. `model="probe"` uses the probe output noise density where the workflow can supply it, which is important when the received spectrum is filtered by a tuned, untuned, or matched resonator. Result objects preserve clean deterministic fields and add noisy fields plus `NoiseMetadata` so repeated SNR studies remain reproducible.

8.6 Diffusion

```
from spin_dynamics.workflows import run_matched_diffusion_q_sweep

sweep = run_matched_diffusion_q_sweep(
    q_values=[20, 50],
    num_echoes=3,
    numpts=21,
```

```
    dz=50e-6,
    auto_refine_grid=True,
)
```

The compact matched-probe diffusion workflow is solver-validated through $Q = 2000$. Higher-Q cases remain a stiffness target for the current NumPy matched-probe transient solver.

Matched diffusion CPMG also accepts `absolute_phase`. In this combined path, the diffusion-encoding π pulse and the CPMG refocusing pulses are solved at their laboratory-frame RF phase, while the free-precession intervals retain the diffusion attenuation:

```
from spin_dynamics.workflows import run_matched_diffusion_cpmg

combined = run_matched_diffusion_cpmg(
    num_echoes=16,
    echo_spacing_seconds=1.0e-3,
    diffusion_time=1.0e-3,
    q_value=50,
    absolute_phase={
        "rf_frequency_hz": 0.25 / 1.0e-3,
        "phase_bins": 16,
    },
)

metadata = combined.absolute_phase
encoding_phase = metadata.encoding_absolute_phase_rad
refocus_phases = metadata.refocus_absolute_phase_rad
```

`metadata.refocus_*` describes the echo-train refocusing pulses. `metadata.encoding_*` describes the diffusion-encoding π pulse, and `pulse_*` fields provide the full RF event list.

The comparison plotting example toggles diffusion and absolute phase independently:

```
python examples\plot_diffusion_absolute_phase_compare.py
python examples\plot_tuned_diffusion_absolute_phase_compare.py
```

The diffusion plotting examples use a narrow default `-dz-um` and enable automatic offset-grid refinement so the displayed decay is not an artifact of discrete-grid rephasing. They print the effective number of offsets after any refinement. Use `-no-auto-refine-grid` only when intentionally testing the rephasing guard. The matched diffusion comparison also prints the matched-probe absolute-phase residual and warns when it is below the plotting tolerance. The current matched pulse-shape solver is often nearly phase-invariant; in that case the plot is a diagnostic limitation rather than evidence for a measurable combined effect. The tuned-probe comparison uses the same diffusion kernel with tuned pulse-shape solves and usually shows a much larger residual, making it the better contrast example for phase-sensitive probe transients.

8.6.1 PGSE and PGSE-Prepared CPMG

For Stejskal-Tanner pulsed-gradient diffusion encoding, use the PGSE workflow helpers. The deterministic backend tracks the effective gradient moment and therefore evaluates

$$b = \int q(t)^2 \, \mathrm{d}t, \qquad q(t) = \gamma \int_0^t G_{\mathrm{eff}}(t') \, \mathrm{d}t'.$$

For a rectangular PGSE pair this reduces to

$$b = (\gamma G \delta)^2 \left(\Delta - \frac{\delta}{3} \right),$$

where δ is the lobe duration and Δ is the leading-edge separation between the two gradient lobes. The physical lobes are scheduled with the same polarity around the refocusing pulse; the 180-degree pulse flips the coherence-frame sign, so stationary spins refocus while diffusive motion attenuates the signal.

```
from spin_dynamics.workflows import run_pgse_moment

pgse = run_pgse_moment(
    num_echoes=8,
    gradient_amplitude=0.12,      # T/m
    gradient_duration=2.5e-3,     # delta
    diffusion_time=28.0e-3,       # Delta
    diffusion_coefficient=1.2e-9,
    first_echo_time_seconds=56e-3,
    echo_spacing_seconds=8e-3,
    t2_seconds=80e-3,
)
```

`num_echoes=1` is the ordinary spin-echo PGSE acquisition. Larger echo counts represent a PGSE preparation followed by a compact CPMG acquisition. For spatially explicit studies, `run_pgse_walkers` uses the moving-isochromat sequence driver and seeded Brownian walkers. It is slower and stochastic, but can be extended to inhomogeneous field maps, boundaries, and advection:

```
from spin_dynamics.workflows import run_pgse_walkers

walkers = run_pgse_walkers(
    gradient_amplitude=0.05,
    gradient_duration=2.0e-3,
    diffusion_time=16.0e-3,
    diffusion_coefficient=2.3e-9,
    walkers_per_cell=12000,
    seed=123,
)
```

The PGSE tests compare the moment backend to the analytical Stejskal-Tanner formula, and compare the walker backend against the same attenuation through a zero-diffusion walker reference.

8.6.2 Restricted Diffusion and Diffusive Diffraction

Because the walker backend integrates explicit displacements, it can model diffusion restricted by hard walls or exchanged through semi-permeable membranes—physics the closed-form moment backend cannot express. The `boundary` argument of `run_pgse_walkers` accepts the rectangular-box modes `"reflect"`, `"periodic"`, and `"clip"`, or any callable mapping particle positions to confined positions. The helpers `make_circular_reflector(center, radius)` and `make_elliptical_reflector(center, semi_axes)` supply reflecting circular and elliptical pore walls.

For a slab pore of width L along the gradient axis, confining the walkers makes the echo attenuation bend above the free Stejskal-Tanner line $E = \exp(-bD)$: once the diffusion length $\sqrt{2D\Delta}$ approaches L the spins can no longer fully dephase, and the apparent diffusion coefficient $D_{\mathrm{app}} = -\ln(E)/b$ falls below the

bulk value D as the diffusion time grows. The example `plot_pgse_restricted_diffusion.py` sweeps several pore widths and diffusion times to show both signatures.

In a closed two-dimensional pore the restricted signal becomes non-monotonic. In the narrow-pulse, long-mixing limit ($\delta \ll a^2/D \ll \Delta$) the normalized echo approaches the squared magnitude of the pore form factor, the *diffusive diffraction* regime of q-space NMR. For a uniform disc of radius a,

$$E(q) \;\longrightarrow\; \left| \frac{2\,J_1(q_{\mathrm{ang}} a)}{q_{\mathrm{ang}} a} \right|^2,$$

with minima at the zeros of J_1, i.e. $q_{\mathrm{ang}} a = 3.83,\ 7.02,\ 10.17, \ldots$. Note the factor-of-$2\pi$ ambiguity in the definition of q: the *angular* wavenumber is $q_{\mathrm{ang}} = \gamma G \delta$ (units $\mathrm{rad/m}$, with $b = q_{\mathrm{ang}}^2(\Delta - \delta/3)$), while the *reciprocal-space* (Callaghan) convention uses $q = \gamma G \delta/(2\pi)$ (units $1/\mathrm{m}$, encoding kernel $\exp(i 2\pi q x)$), for which the first disc feature sits near $qa \sim 1$.

```
import numpy as np
from spin_dynamics.motion import make_circular_reflector
from spin_dynamics.workflows import run_pgse_walkers

radius = 5.0e-6
axis = np.linspace(-radius, radius, 21)
xx, zz = np.meshgrid(axis, axis, indexing="ij")
rho = (xx**2 + zz**2 <= radius**2).astype(float)  # uniform disc

walkers = run_pgse_walkers(
    rho=rho, x_axis=axis, z_axis=axis,
    gradient_amplitude=2.0,        # large G reaches the q-space regime
    gradient_duration=0.4e-3,      # short delta (narrow-pulse limit)
    diffusion_time=80.0e-3,        # long Delta >> a^2 / D
    diffusion_coefficient=2.3e-9,
    boundary=make_circular_reflector((0.0, 0.0), radius),
    walkers_per_cell=28, substeps_per_interval=80, seed=2026,
)
```

The example `plot_pgse_circular_pore_diffraction.py` plots the resulting fringes against the Bessel form factor. It offers two methods. The default `walker-sweep` re-runs the confined PGSE simulation at every gradient and so captures finite-pulse blurring of the minima. The optional `-method sgp-propagator` exploits the fact that the walker trajectories are independent of q: it runs the confined diffusion once, records each walker's net displacement Δx over the diffusion time, and evaluates $E(q) = \left|\langle \exp(i q_{\mathrm{ang}} \Delta x) \rangle\right|$ for all q in a single vectorized sum. This is the average-propagator (q-space) method; it is tens of times faster and reproduces the same diffraction pattern in the $\delta \to 0$ limit. Keep the per-substep hop $\sqrt{2D\,\mathrm{d}t}$ well below the pore size so reflection stays accurate.

For slow exchange between two compartments, use `make_semipermeable_plane`. The membrane is an internal line $x = x_m$ or $z = z_m$ inside the rectangular bounds. A walker that crosses the line transmits with probability

$$P_{\mathrm{transmit}} = 1 - \exp(-k\,\Delta t),$$

where k is `exchange_rate`; otherwise the walker reflects from the membrane. This rate form stays consistent when `substeps_per_interval` is changed.

```
import numpy as np
from spin_dynamics.motion import make_semipermeable_plane
from spin_dynamics.workflows import run_pgse_walkers
```

```
x = np.linspace(-10e-6, 10e-6, 41)
z = np.array([-0.5e-6, 0.5e-6])
rho = np.ones((x.size, z.size))
membrane = make_semipermeable_plane(
    0.0,
    exchange_rate=25.0,  # s^-1 in this seconds-based workflow
    axis="x",
)

walkers = run_pgse_walkers(
    rho=rho, x_axis=x, z_axis=z,
    gradient_amplitude=0.2, gradient_duration=1.0e-3,
    diffusion_time=40.0e-3,
    diffusion_coefficient=2.0e-9,
    boundary=membrane,
    walkers_per_cell=64, substeps_per_interval=16, seed=2026, jitter=True,
)
```

Use `exchange_rate=0` for an impermeable internal wall and `np.inf` for a freely transmitting interface. The outer rectangular boundary still defaults to reflection, so the exchanging compartments remain confined by the sample box.

8.6.3 Stimulated-Echo PGSE (PGSTE)

`run_pgste_walkers` splits the diffusion encoding across three 90-degree pulses, $90 - G(\delta) - 90 - [\text{storage}] - 90 - G(\delta) - \text{echo}$. The second pulse stores one quadrature of the encoded magnetization along the longitudinal axis, so during the (long) storage interval it decays with T_1 rather than T_2. This is why PGSTE is the method of choice for slow diffusion, large pores, and short-T_2 samples such as porous media, low field, and internal-gradient regimes. A spoiler gradient applied during storage crushes the residual transverse coherences. The workflow records an effective selected-pathway phase table and suppresses equilibrium recovery into a contaminating FID, so the surviving stimulated echo follows the Stejskal-Tanner attenuation at *half* the spin-echo amplitude:

$$E(b) \;\longrightarrow\; \tfrac{1}{2}\exp\!\left(-\frac{T_s}{T_1}\right)\exp(-bD),$$

where T_s is the storage interval. The argument `diffusion_time` is the leading-edge separation of the two gradient lobes, so $b = (\gamma G\delta)^2(\Delta - \delta/3)$ still holds, and $T_s = \Delta - \delta - 2\,\texttt{encode\_delay} - 2\,\texttt{excitation\_duration}$.

```
import numpy as np
from spin_dynamics.workflows import run_pgste_walkers

axis = np.linspace(-1.0e-3, 1.0e-3, 64)   # wide slab (see note below)
rho = np.ones((axis.size, 2))

ste = run_pgste_walkers(
    rho=rho, x_axis=axis, z_axis=np.array([-1e-6, 1e-6]),
    gradient_amplitude=0.1, gradient_duration=1.0e-3,  # delta
    diffusion_time=60.0e-3,            # Delta = lobe leading-edge separation
    diffusion_coefficient=2.3e-9,
    t1_seconds=0.4, t2_seconds=15.0e-3,  # storage decays with T1, not T2
    walkers_per_cell=64, seed=2026, jitter=True,
)
ste.phase_cycle.name   # "pgste_stimulated_echo"
```

Use a sample wide compared with the gradient phase wavelength $\pi/(\gamma G\delta)$ so the unwanted stimulated anti-echo dephases spatially; with diffusion present it is suppressed further. The example `plot_pgste_stimulated_echo.py` verifies the half-amplitude Stejskal-Tanner attenuation and contrasts the T_2-limited spin echo with the T_1-limited stimulated echo as the diffusion time grows.

8.6.4 Double Diffusion Encoding (DDE / Double-PGSE)

`run_dde_walkers` applies two refocused PGSE blocks separated by a mixing time, $90 - [G_1\,\text{block}] - \text{mixing} - [G_2\,\text{block}] - \text{echo}$, encoding displacement along `angle1` and `angle2`. Single-PGSE measures the diffusion tensor, so in an orientationally disordered sample of anisotropic compartments it sees only the isotropic orientation average. Sweeping the angle ψ between the two encoding directions instead produces a $\cos 2\psi$ modulation of the echo whose amplitude reports the *microscopic* anisotropy of the local geometry; because it depends only on the relative angle, it survives powder averaging and reveals shape anisotropy that single-PGSE cannot. Paired with `make_elliptical_reflector`, an elliptical pore shows the $\cos 2\psi$ term while an equal-area circular pore does not. The $\cos 2\psi$ term is a higher-order (q^4) effect, so strong diffusion weighting is needed to resolve it in the powder average.

```
import numpy as np
from spin_dynamics.motion import make_elliptical_reflector
from spin_dynamics.workflows import run_dde_walkers

semi_axes = (8.0e-6, 3.0e-6)
x = np.linspace(-semi_axes[0], semi_axes[0], 15)
z = np.linspace(-semi_axes[1], semi_axes[1], 15)
xx, zz = np.meshgrid(x, z, indexing="ij")
rho = ((xx / semi_axes[0]) ** 2 + (zz / semi_axes[1]) ** 2 <= 1.0).astype(float)

dde = run_dde_walkers(
    rho=rho, x_axis=x, z_axis=z,
    gradient_amplitude=1.0, gradient_duration=1.0e-3,
    diffusion_time=12.0e-3, mixing_time=1.0e-3,
    angle1=0.0, angle2=np.pi / 2,          # vary to trace E(psi)
    diffusion_coefficient=2.0e-9,
    boundary=make_elliptical_reflector((0.0, 0.0), semi_axes),
    walkers_per_cell=64, substeps_per_interval=10, seed=2026, jitter=True,
)
```

The example `plot_pgse_double_encoding_elliptical_pore.py` traces $E(\psi)$ for the ellipse and the equal-area circle, both at a single orientation and powder-averaged, and reports the fitted $\cos 2\psi$ amplitude.

8.6.5 Oscillating-Gradient Spin Echo (OGSE)

PGSE varies the diffusion time Δ; `run_ogse_walkers` instead varies the oscillation frequency of a cosine-modulated gradient. Each diffusion-encoding lobe is a waveform $G\cos(2\pi f t)$ of `num_periods` whole periods, applied symmetrically around the refocusing pulse, $90 - \cos$ lobe $- 180 - \cos$ lobe $-$ echo. Because each lobe spans an integer number of periods, its zeroth gradient moment vanishes and stationary spins refocus. The encoding power is concentrated at the angular frequency $\omega = 2\pi f$, so sweeping `oscillation_frequency` maps the diffusion spectrum $D(\omega)$ and reaches the short-diffusion-time regime that ordinary PGSE cannot. The waveform is discretized into `samples_per_period` constant-gradient steps, and the b-value follows the cosine spectrum $b = (\gamma G/\omega)^2\, N/f$, which falls steeply with frequency.

```
import numpy as np
from spin_dynamics.motion import make_motion_field_maps_2d
from spin_dynamics.workflows import run_ogse_walkers

x = np.linspace(-2.5e-6, 2.5e-6, 15)         # 5 um reflecting slab along x
z = np.array([-0.5e-6, 0.5e-6])
rho = np.ones((x.size, z.size))

ogse = run_ogse_walkers(
    rho=rho, x_axis=x, z_axis=z,
    fields=make_motion_field_maps_2d(x, z),  # walls = the slab
    gradient_amplitude=0.5, oscillation_frequency=200.0,  # sweep this
    num_periods=2, diffusion_coefficient=2.0e-9,
    walkers_per_cell=160, substeps_per_interval=6, seed=2026, jitter=True,
)
d_app = -np.log(abs(ogse.signal[0]) / rho.sum()) / ogse.b_value  # D(omega)
```

In restricted geometry D_{app} rises from the long-time (tortuosity) value toward the bulk value as the frequency increases, and a smaller pore stays restricted to higher frequency (transition $f_c \sim D/L^2$). The example `plot_ogse_frequency_diffusion.py` sweeps the frequency for free diffusion and two reflecting slab widths.

8.7 Moving Isochromats

The motion layer treats inhomogeneity in the same spirit as static isochromats, but the samples move through two-dimensional B0, transmit-B1, and receive-B1 maps:

```
from spin_dynamics.motion import make_motion_field_maps_2d,
    initialize_ensemble_from_density
from spin_dynamics.sequences.motion import (
    run_motion_cpmg_sequence,
    run_motion_udd_sequence,
)

fields = make_motion_field_maps_2d([-1, 0, 1], [-1, 0, 1])
ensemble = initialize_ensemble_from_density([[0, 1, 0], [1, 2, 1], [0, 1, 0]])
motion = run_motion_cpmg_sequence(
    ensemble,
    fields,
    num_echoes=4,
    echo_spacing=1e-3,
    excitation_duration=20e-6,
    refocusing_duration=40e-6,
    gradient=(0.0, 0.1),
)

udd = run_motion_udd_sequence(
    ensemble,
    fields,
    num_pulses=4,
    total_duration=4e-3,
    excitation_duration=20e-6,
    refocusing_duration=40e-6,
```

```
    gradient=(0.0, 0.1),
)
```

Lower-level helpers such as `free_precession_with_motion_step` are useful for custom trajectories. The sequence helpers split long intervals into substeps so field maps are sampled along the particle path rather than only at the beginning of an interval.

The motion sequence runners also accept `detuning_waveform`. The callable may return a scalar B0 offset for the whole ensemble or one value per particle. This makes it possible to model slow fluctuating gradients or local bath fields:

```
import numpy as np

fluctuating_gradient = lambda time, positions: (
    1500.0 * positions[:, 0] * np.cos(2 * np.pi * 0.35 * time)
)

udd = run_motion_udd_sequence(
    ensemble,
    fields,
    num_pulses=8,
    total_duration=0.72,
    excitation_duration=0.25e-3,
    refocusing_duration=0.5e-3,
    t1=5.0,
    t2=1.0,
    detuning_waveform=fluctuating_gradient,
    substeps_per_interval=8,
)
```

This is the more realistic regime where UDD can outperform evenly spaced CPMG: the noise is low-frequency and correlated across time, but varies across the sample so residual phase spread reduces the received signal. In contrast, ordinary unrestricted diffusion in a static gradient usually gives very similar CPMG and UDD attenuation.

8.8 Time-Varying Fields

```
from spin_dynamics.workflows import (
    sinusoidal_field_waveform,
    run_ideal_time_varying_amplitude_sweep,
)

waveform = sinusoidal_field_waveform(num_echoes=16)
sweep = run_ideal_time_varying_amplitude_sweep(
    amplitudes=[0, 0.5, 1.0, 2.0],
    waveform=waveform,
    numpts=101,
    auto_refine_grid=True,
)
```

8.9 WURST

```
from spin_dynamics.workflows import (
    run_ideal_wurst_inversion,
    run_matched_wurst_cpmg,
)

ideal = run_ideal_wurst_inversion(numpts=101)
matched = run_matched_wurst_cpmg(num_echoes=2, numpts=51, num_steps=64)
```

8.10 Phase Cycling

The Python API represents phase cycling with `spin_dynamics.phase_cycling.PhaseCycle`. A phase cycle is a scan table: rows are cycle steps, and columns are named logical RF pulse roles or events. For example, the default CPMG/PAP table has one pulse column, `excitation`, and two rows with phases $\pi/2$ and $3\pi/2$. Each row also carries a receiver phase, a branch weight, and an optional label. Branches are combined as

$$\sum_k w_k \exp(-i\phi_{\mathrm{rx},k})\, S_k,$$

with normalization by the sum of absolute receiver-weighted branch weights unless the table disables normalization.

```
from spin_dynamics.phase_cycling import cpmg_two_step_phase_cycle
from spin_dynamics.workflows import run_ideal_cpmg_train

cycle = cpmg_two_step_phase_cycle()
cycle.pulse_names        # ("excitation",)
cycle.branch_weights     # array([0.5+0.j, -0.5+0.j])

train = run_ideal_cpmg_train(num_echoes=8, numpts=51)
train.phase_cycle.name   # "cpmg_two_step"
```

The phase-cycle columns are not the set of unique RF phase values. They are the named pulse roles whose phase can vary from one scan branch to the next. This keeps the table aligned with sequence events and leaves room for receiver phase and weighting information on each row.

PGSTE walker results expose the same metadata field, but the interpretation is more specific. The `pgste_` constructor returns a one-row table for the selected stimulated-echo pathway with columns `excitation_90`, `store_90`, and `read_90`. The PGSTE workflow does not explicitly simulate and receiver-combine several phase-cycle branches; pathway selection is implemented by the spoiler and by suppressing equilibrium regrowth during storage.

`plot_phase_cycled_stimulated_echo.py` shows the explicit three-pulse phase table on a deterministic inhomogeneous-field ensemble. It compares a single three-90° scan, a read-pulse-only two-step cycle, and a full 64-step table over `excitation_90`, `store_90`, and `read_90`. During storage the example applies physical relaxation: transverse magnetization decays with T_2, while M_z relaxes back toward equilibrium with T_1. The recovered M_z produces a prompt last-pulse FID, and the full receiver signature $-\phi_1 + \phi_2 + \phi_3$ rejects that pathway while retaining the stimulated echo at τ.

8.11 Absolute RF Phase

Finite CPMG train workflows accept an optional `absolute_phase` mapping. The ideal workflow records the laboratory-frame phase schedule while preserving its nominal rectangular pulses. Probe-aware finite trains use the same schedule to solve the tuned, un-tuned, or matched probe waveform for each pulse, discretize the rotating-frame waveform into short segments, and build a separate refocusing matrix for each echo:

```
from spin_dynamics.workflows import run_tuned_cpmg_train

train = run_tuned_cpmg_train(
    numpts=21,
    num_echoes=32,
    absolute_phase={
        "rf_frequency_hz": 0.25 / 200e-6,
        "rf_phase_at_zero_rad": 0.0,
    },
)

phase_step = train.absolute_phase.delta_refocus_phase_cycles
```

For long trains, add `phase_bins` to reuse solved refocusing pulses on a uniform absolute-phase grid:

```
train = run_tuned_cpmg_train(
    num_echoes=256,
    absolute_phase={
        "rf_frequency_hz": 0.03125 / 200e-6,
        "phase_bins": 32,
    },
)

library = train.absolute_phase.refocus_pulse_library
matrix_phases = train.absolute_phase.refocus_matrix_phase_rad
```

The scheduled phase for every echo remains available as `refocus_absolute_phase_rad`. The quantized matrix phase, bin index, unique phase list, and exported `PulseShapeLibrary` are recorded in result metadata so cache behavior can be audited or reused in downstream analysis. The matched diffusion CPMG workflow uses the same `absolute_phase` mapping and additionally records the diffusion-encoding π pulse in `encoding_*` metadata fields.

The same solved pulse shapes can be inspected without running a full echo train:

```
import numpy as np

from spin_dynamics.pulse_diagnostics import solve_probe_pulse_shape_sweep

sweep = solve_probe_pulse_shape_sweep(
    probe="tuned",
    absolute_phase_rad=2 * np.pi * np.array([0.0, 0.25, 0.5, 0.75]),
    numpts=17,
)

shape = sweep.shapes[1]
drive = shape.drive
quadrature_fraction = shape.quadrature_energy_fraction
library = sweep.pulse_shape_library
```

When sweeping the phase advance between refocusing pulses, keep the absolute phase of the first refocusing pulse fixed if the goal is to isolate echo-to-echo variation. A half-cycle phase advance produces the same transverse rotating-frame pulse shape for a linear tuned probe, while intermediate advances change the rise/fall and quadrature transients and can produce extra attenuation in the matched-filter echo amplitude.

The Mandal-2015 plotting examples expose the diagnostic workflow directly:

```
python examples\plot_mandal2015_phase_step_sweep.py --probe tuned
python examples\plot_mandal2015_echo_modulation.py --probe tuned
python examples\plot_mandal2015_pulse_shapes.py --probe tuned
```

The finite-train plotting scripts accept `-phase-bins N` to exercise the same quantized pulse-cache path used by the workflow API. They also enable automatic offset-grid refinement by default and use `-rephase-action raise`; this keeps demonstration plots from silently using too few isochromats. Pass `-no-auto-refine-grid` only for explicit diagnostic runs.

`plot_mandal2015_pulse_shapes.py` solves a single refocusing pulse at several absolute RF phases and plots $\mathrm{Re}(B_1)$, $\mathrm{Im}(B_1)$, $|B_1|$, and the phase of the nonzero-amplitude segments. It is intended as a debugging and teaching aid: the finite train examples use the same public diagnostics API, pulse-shape solvers, and discretized segments.

8.12 Radiation Damping

```
from spin_dynamics.workflows import run_radiation_damping_fid

fid = run_radiation_damping_fid(
    probe="matched",
    fill_factor=0.7,
    equilibrium_magnetization=0.8,
    flip_angle=1.0,
    model="circuit",
    detuning=2.0e4,
)
```

Finite tuned and matched CPMG trains accept an opt-in `radiation_damping` mapping:

```
from spin_dynamics.workflows import run_tuned_cpmg_train

train = run_tuned_cpmg_train(
    numpts=51,
    num_echoes=8,
    radiation_damping={
        "fill_factor": 0.7,
        "equilibrium_magnetization": 0.8,
        "model": "circuit",
        "detuning": 2.0e4,
        "apply_during_pulses": True,
    },
)
```

The CPMG train path is deterministic and keeps the radiation-damping feedback inside the finite acquisition model. For strongly inverted samples, `simulate_nmr_maser` uses the same feedback machinery with a pumped longitudinal magnetization; it is intended as an idealized threshold and saturation diagnostic rather than a full spectrometer-control model.

8.13 Inverse Laplace Analysis

```
import numpy as np
from spin_dynamics.analysis import invert_t2, t2_kernel

echo_times = np.linspace(0.5e-3, 90e-3, 40)
t2_axis = np.logspace(-4, -1, 60)
signal = t2_kernel(echo_times, t2_axis) @ np.exp(-((np.log(t2_axis) + 4.5) ** 2))

result = invert_t2(
    signal,
    echo_times,
    t2_axis,
    regularization=5e-4,
    regularization_order=2,
)
```

For two-dimensional distributions, the separable forward model is

$$D = K_1 F K_2^{\mathsf{T}},$$

where D is the measured data, F is the unknown distribution, and K_1, K_2 are relaxation or diffusion kernels. Use `invert_t1_t2` or `invert_d_t2` for common NMR cases.

The PGSE D-T2 example ties this analysis layer to a sequence model: it builds the b-axis with `run_pgse_moment`, simulates a PGSE-prepared CPMG echo matrix, and recovers the D-T2 distribution with the SciPy-backed non-negative `invert_d_t2` solver.

8.14 Chemical and Site Exchange

The `spin_dynamics.exchange` namespace adds Bloch-McConnell site exchange, the relaxation-domain counterpart to the diffusion-exchange (DEXSY) walker example. An `ExchangeSystem` holds a tuple of `ExchangeSite` objects and a rate matrix whose entry (i, j) for $i \neq j$ is the first-order rate constant $k_{i\to j}$. Site populations are normalized, and detailed balance $p_i k_{i\to j} = p_j k_{j\to i}$ is checked unless disabled.

```
import numpy as np
from spin_dynamics.exchange import (
    two_site_exchange, simulate_relaxation_exchange_2d, exchange_spectrum,
)
from spin_dynamics.analysis import invert_t2_t2

system = two_site_exchange(
    offset_a_hz=0.0, offset_b_hz=0.0,
    k_ab_hz=8.0, population_a=0.5,
    t2_a_seconds=0.010, t2_b_seconds=0.200,
    labels=("fast", "slow"),
)
```

The kinetic generator X ($\dot{m} = Xm$) is column-conserving, so exchange alone preserves total magnetization. Adding per-site offset precession and transverse relaxation gives the transverse generator used for lineshapes: `exchange_spectrum` integrates the transverse Bloch-McConnell equations and reproduces the slow-exchange (two resolved lines) to fast-exchange (a single population-averaged line) coalescence crossover.

For relaxation exchange spectroscopy (REXSY), the encode-mix-detect model places exchange in a longitudinal mixing interval described by a mixing propagator G:

$$S(t_1, t_2) = \sum_b e^{-t_2/T_{2,b}} \sum_a G_{ba}\, p_a\, e^{-t_1/T_{2,a}}.$$

Inverting $S(t_1, t_2)$ with `invert_t2_t2` yields a T_2-T_2 map: diagonal peaks for spins whose T_2 is unchanged across mixing, and off-diagonal cross peaks whose intensity grows with the exchange rate.

```
encode = np.linspace(0.0, 0.06, 28)
detect = np.linspace(0.0, 0.8, 28)
rexsy = simulate_relaxation_exchange_2d(system, encode, detect, mixing_time=0.06)

t2_axis = np.logspace(-3, 0, 48)
ilt = invert_t2_t2(rexsy.data, encode, detect, t2_axis, regularization=1e-3)
exchange_map = ilt.distribution  # off-diagonal mass == exchange
```

The `plot_t2_t2_exchange.py` example runs this end to end. The REXSY forward model assumes refocused encode/detect trains and exchange confined to the longitudinal mixing interval; use the transverse engine directly when offset evolution during encoding matters.

8.15 Internal and Susceptibility Gradients

The `spin_dynamics.susceptibility` namespace generates the static internal field produced by magnetic-susceptibility contrast between a solid matrix and the pore fluid, the inhomogeneity that usually dominates porous-media NMR. Each `CylindricalInclusion` is an infinitely long cylinder perpendicular to the two-dimensional map plane, magnetized by a uniform in-plane applied field. Outside the cylinder the parallel field perturbation is the 2D dipole

$$\Delta B_{\parallel}(\rho, \varphi) = B_0 \frac{\Delta\chi}{2} \left(\frac{a}{\rho}\right)^2 \cos(2\varphi),$$

with radius a, distance ρ, and angle φ measured from the applied-field direction. Cylinders superpose in the dilute ($|\Delta\chi| \ll 1$) limit.

```
import numpy as np
from spin_dynamics.susceptibility import (
    CylindricalInclusion, susceptibility_offresonance_map,
    internal_gradient_distribution, make_susceptibility_field_maps,
)

axis = np.linspace(-50e-6, 50e-6, 161)
grains = [CylindricalInclusion(cx, cz, 12e-6)
          for cx in (-30e-6, 30e-6) for cz in (-30e-6, 30e-6)]

field = susceptibility_offresonance_map(
    axis, axis, grains, b0_tesla=2.0, susceptibility_difference=1e-6,
)
dist = internal_gradient_distribution(field)  # pore-space |g| in T/m
maps = make_susceptibility_field_maps(field)  # drop into motion helpers
```

The off-resonance map is returned in the angular-frequency (rad/s) convention used by `spin_dynamics.motion`, so it feeds the moving-isochromat sequence helpers directly. With no applied gradient, a CPMG echo train of diffusing walkers then decays purely from diffusion in the internal gradient.

The acceleration of that decay with echo spacing (rate $\sim G^2 D\, T_E^2$) is the general diffusion-in-a-gradient effect and is *not* by itself a signature of an internal gradient: a uniform applied or static gradient produces it too. What distinguishes a susceptibility-induced gradient is its field dependence. Because $G_{\mathrm{internal}} \propto \Delta\chi\, B_0$, the diffusion-induced decay rate scales as B_0^2, whereas an applied gradient is set by hardware and is independent of B_0. The broad pore-space gradient *distribution* is a second distinguishing feature; a uniform gradient would be a single value. The `plot_internal_gradients.py` example shows the internal field, the internal-gradient distribution, and the recovered B_0^2 scaling of the diffusion-induced decay rate end to end. Background-gradient-suppressing sequences are implemented in the next section; the internal field produced here is their input.

8.16 Bipolar 13-Interval PGSTE

The `spin_dynamics.workflows.bipolar` module implements the Cotts 13-interval alternating pulsed-gradient stimulated-echo (APGSTE) sequence, the sequence in the Bruker `diff_stebp.gp` program, which measures diffusion while suppressing a constant background gradient g_0 such as the internal gradient of Section 8.15. It is a stimulated echo (three 90 degree pulses) with one 180 degree pulse in each of the two encoding periods, a gradient lobe on each side of every 180, and the applied polarity inverted between the two periods. The 180 pulses refocus the background gradient within each period, and the polarity alternation cancels the applied-times-background cross-term.

Working in the toggling (coherence) frame, the diffusion attenuation in a constant background gradient is

$$\ln E = -D\left(b_{\mathrm{applied}} + g_0\, c_{\mathrm{cross}} + g_0^2\, c_{\mathrm{bg}}\right),$$

and `toggling_frame_moments` returns the three coefficients. For the 13-interval sequence $c_{\mathrm{cross}} = 0$; for an ordinary monopolar stimulated echo it is non-zero, so its apparent diffusion coefficient drifts with g_0.

```
from spin_dynamics.workflows import (
    run_cotts_thirteen_interval_moment, run_monopolar_pgste_moment,
)

c = run_cotts_thirteen_interval_moment(gradient_amplitude=0.1, background_gradient=0.05)
m = run_monopolar_pgste_moment(gradient_amplitude=0.1, background_gradient=0.05)
print(c.cross_term_bias, m.cross_term_bias)  # ~0 vs non-zero
print(c.phase_cycle.name)                   # "diff_stebp"
```

The phase cycle is the 16-step Bruker `diff_stebp` table, built on the package's `PhaseCycle` machinery as `diff_stebp_phase_cycle()`. Its receiver satisfies $\mathrm{ph31} = -(\mathrm{ph1} + \mathrm{ph2} + \mathrm{ph3}) \bmod 4$ (with $\mathrm{ph4} = 0$ on both 180 pulses), selecting the stimulated-echo coherence-transfer pathway $\Delta p = (+1, -2, +1, +1, -2)$ with detection at $p = -1$; the coherence order $p(t)$ this cycle selects is exactly the sign carried by the toggling-frame intervals. The `plot_bipolar_pgste.py` example traces the wavevectors and the apparent-diffusion suppression versus g_0. The moment model is the free-diffusion b-value for ideal rectangular lobes; trapezoidal ramps and restricted geometry need the walker pipeline.

8.17 Optimization Workflows

The optimization package provides compact, plotting-free drivers for phase-program searches and result handoff:

```
from spin_dynamics.optimization import run_tuned_refocusing_multistart

opt = run_tuned_refocusing_multistart(
    num_segments=3,
    num_starts=4,
    numpts=21,
    max_passes=4,
)
```

Use the optimization examples as diagnostics rather than turnkey production pulse design: they expose refocusing, target-excitation, and inverse-excitation objective behavior while the inverse-cancellation workflow remains under validation.

9 Examples

The example scripts can be run from `PythonSpinDynamics`; each script also works from inside `examples/` by adding the local source tree to `sys.path` when needed.

```
python examples\ideal_cpmg.py --numpts 101
python examples\ideal_fid.py --numpts 101
python examples\ideal_cpmg_train.py --numpts 101 --num-echoes 8
python examples\ideal_time_varying_cpmg.py --numpts 101 --num-echoes 16
python examples\compare_cpmg_fid.py --numpts 101
python examples\export_validation_arrays.py results\validation_arrays.npz --numpts 101
python examples\tuned_probe_cpmg.py --numpts 101
python examples\probe_cpmg_compare.py --numpts 101
python examples\probe_parameter_sweeps.py --numpts 101
python examples\matched_cpmg_ir_train.py --numpts 21 --num-echoes 4 --num-tau 4
python examples\finite_probe_train_sweeps.py --numpts 21 --num-echoes 3
python examples\matched_diffusion_cpmg.py --numpts 21 --num-echoes 3
python examples\received_signal_noise.py --numpts 51
python examples\radiation_damping_fid.py --probe matched --points 401
python examples\radiation_damping_cpmg_train.py --probe tuned --numpts 21 --num-echoes 4
python examples\nmr_maser.py
python examples\heteronuclear_j_editing.py --points 33
python examples\coupled_isochromat_fields.py --points 21
python examples\diagnose_optimization_backends.py --backend all --numpts 21 --segments 3
```

Plotting examples require Matplotlib:

```
python examples\plot_ideal_workflows.py --numpts 201 --output results\ideal_workflows.png
python examples\plot_ideal_imaging.py --pixels 6 --ny 7 --output results\ideal_imaging.png
python examples\plot_custom_imaging_fields.py --pixels 8 --ny 7 --output results\
    custom_imaging_fields.png
python examples\plot_inverse_laplace.py --output results\inverse_laplace.png
python examples\plot_pgse_d_t2.py --output results\pgse_d_t2.png
python examples\plot_phase_cycled_stimulated_echo.py --output results\phase_cycled_ste.png
python examples\plot_probe_parameter_sweep.py --probe tuned --sweep q --output results\
    tuned_q_sweep.png
python examples\plot_probe_cpmg.py --numpts 101 --output results\probe_cpmg.png
python examples\plot_finite_train_workflows.py --numpts 65 --num-echoes 4 --output results
    \finite_trains.png
python examples\plot_diffusion_sweep.py --numpts 65 --num-echoes 3 --output results\
    diffusion_sweep.png
python examples\plot_diffusion_absolute_phase_compare.py --output results\
    diffusion_absolute_phase_compare.png
python examples\plot_tuned_diffusion_absolute_phase_compare.py --output results\
    tuned_diffusion_absolute_phase_compare.png
python examples\plot_time_varying_sweep.py --numpts 51 --num-echoes 12 --output results\
    time_varying_sweep.png
python examples\plot_udd_cpmg_filter.py --pulses 8 --output results\udd_cpmg_filter.png
python examples\plot_motion_linear.py --output results\motion_linear.png
python examples\plot_motion_diffusion_cpmg.py --output results\motion_diffusion_cpmg.png
python examples\plot_motion_diffusion_udd.py --output results\motion_diffusion_udd.png
python examples\plot_radiation_damping.py --output results\radiation_damping.png
python examples\plot_radiation_damping_detuning.py --output results\rd_detuning.png
python examples\plot_radiation_damping_cpmg_train.py --output results\rd_cpmg.png
```

```
python examples\plot_mandal2015_phase_step_sweep.py --probe tuned --output results\
    mandal_phase_sweep.png
python examples\plot_mandal2015_echo_modulation.py --probe tuned --output results\
    mandal_echo_modulation.png
python examples\plot_mandal2015_pulse_shapes.py --probe tuned --output results\
    mandal_pulse_shapes.png
python examples\plot_nmr_maser.py --output results\nmr_maser.png
python examples\plot_wurst_flow.py --numpts 41 --num-steps 64 --output results\wurst_flow.
    png
python examples\plot_j_editing_spectrum.py --output results\j_editing_spectrum.png
python examples\plot_j_editing_field_spread.py --output results\j_editing_field_spread.png
python examples\plot_tango_filter.py --target 160 --output results\tango_filter.png
python examples\plot_slic_two_spin.py --j-hz 7 --delta-hz 0.7 --output results\
    slic_two_spin.png
python examples\plot_optimization_workflows.py --numpts 11 --segments 2 --starts 2 --
    inverse-starts 4 --output results\optimization_workflows.png
python examples\plot_optimization_pipeline.py --numpts 11 --refocusing-segments 2 --
    refocusing-starts 2 --excitation-segments 2 --excitation-starts 2 --inverse-starts 3 --
    output results\optimization_pipeline.png
python examples\plot_nqr_powder_nutation.py --output results\nqr_powder_nutation.png
python examples\plot_nqr_full_powder_nutation.py --output results\nqr_full_powder_nutation.
    png
python examples\plot_nqr_auto_model_selection.py --output results\nqr_auto_model_selection.
    png
python examples\plot_nqr_population_transfer.py --output results\nqr_population_transfer.
    png
python examples\plot_nqr_slse_offset.py --output results\nqr_slse_offset.png
python examples\plot_nqr_slse_spacing.py --output results\nqr_slse_spacing.png
python examples\plot_nqr_efg_broadening.py --output results\nqr_efg_broadening.png
python examples\plot_nqr_temperature_broadening.py --output results\
    nqr_temperature_broadening.png
python examples\plot_nqr_slse_efg_broadening.py --output results\nqr_slse_efg_broadening.
    png
python examples\plot_nqr_weak_b0_spectrum.py --output results\nqr_weak_b0_spectrum.png
```

10 Validation

Validation fixtures live in `validation/fixtures`. The tests compare the Python implementation against compact MATLAB or Octave arrays for core rotations, echo conversion, FID, the `arb10` kernel, finite acquisition, probe response, imaging, pulse-shape helpers, optimization result handling, and public workflow result shapes.

Regenerate dependency-light fixtures with Octave or MATLAB from the `PythonSpinDynamics` directory:

```
matlab -batch "run('validation/octave/generate_basic_fixtures.m')"
matlab -batch "run('validation/octave/generate_imaging_fixtures.m')"
matlab -batch "run('validation/octave/generate_optimization_fixtures.m')"
matlab -batch "run('validation/octave/generate_pulse_fixtures.m')"
```

Matched-probe fixtures require MATLAB toolboxes used by the reference implementation. Octave fixture generation skips matched-probe files when `fmincon` is unavailable.

11 API Reference

This chapter documents the intended public surface. Lower-level module imports remain available for validation and porting. For application code, start with:

```
spin_dynamics.workflows
spin_dynamics.parameters
spin_dynamics.analysis
spin_dynamics.coupling
spin_dynamics.nqr
spin_dynamics.noise
spin_dynamics.motion
spin_dynamics.sequences.motion
spin_dynamics.optimization
```

Drop into lower-level modules only when a building block is needed directly. Use `spin_dynamics.coupling` for the scoped dense scalar-coupled spin-1/2 tools described in Chapter 5. Use `spin_dynamics.nqr` for the selective pulsed NQR tools described in Chapter 7.

11.1 Workflow Results

Class	Purpose
`CPMGResult`	Asymptotic CPMG spectrum, echo, time vector, SNR, probe label, and phase-cycle metadata.
`CPMGTrainResult`	Finite echo-train spectra, echoes, echo integrals, sequence timing, and phase-cycle metadata.
`CPMGIRTrainResult`	Inversion-recovery finite trains over a `tauvect`.
`MatchedCPMGIRTrainResult`	Matched-probe specialization of the CPMG-IR train result.
`CPMGParameterSweepResult`	Asymptotic Q or mistuning sweeps.
`CPMGFiniteParameterSweepResult`	Finite-train Q or mistuning sweeps.
`ZMagnetizationSweepResult`	Matched-probe z-magnetization Q sweeps.
`IdealTimeVaryingCPMGResult`	Final echo for ideal time-varying-field CPMG.
`ProbeTimeVaryingCPMGResult`	Probe-aware time-varying-field CPMG result.
`IdealCPMGImagingResult`	Ideal phase-encoded CPMG imaging result.
`ProbeCPMGImagingResult`	Tuned or matched phase-encoded CPMG imaging result.
`FrequencyEncodedImagingResult`	Spin-warp / RARE frequency-encoded image, k-space, and per-line echo times.
`SliceSensitivityResult`	Real-space sensitive slice (excited/refocused, B1-weighted) of an excitation in a non-uniform field.
`ImagingEchoFitResult`	Voxel-wise mono-exponential echo-stack fit.
`ImagingNoiseStatistics`	Repeated-trial image-domain noise summary.
`MotionSequenceResult`	Moving-isochromat sequence samples and final particle ensemble.

`NoiseMetadata`	Generated received-noise parameters and realized SNR.
`MatchedDiffusionCPMGResult`	Matched diffusion-aware CPMG train.
`MatchedDiffusionQSweepResult`	Matched diffusion Q sweep.
`PGSEMomentResult`	Deterministic PGSE or PGSE-prepared CPMG attenuation from gradient moments.
`PGSEWalkerResult`	Random-walker PGSE signal, sequence samples, and initial particle ensemble.
`PGSTEWalkerResult`	Random-walker stimulated-echo (PGSTE) signal, storage interval, selected-pathway phase-cycle metadata, and initial particle ensemble.
`DDEWalkerResult`	Random-walker double diffusion encoding (DDE) signal, encoding angles, and mixing time.
`OGSEWalkerResult`	Random-walker oscillating-gradient spin-echo (OGSE) signal, oscillation frequency, and b-value.
`BipolarPGSTEResult`	Cotts 13-interval bipolar PGSTE moment model: b-value, background cross-term bias, and the `diff_stebp` phase cycle.
`WURSTInversionResult`	WURST inversion profile and pulse metadata.
`MatchedWURSTCPMGResult`	Matched WURST-CPMG echoes and spectra.
`RadiationDampingFIDResult`	Radiation-damping FID simulation and analytic comparison fields.
`MultiStartOptimizationResult`	Repeated random-start pulse-optimization result.
`RefocusingOptimizationResult`	Bounded fixed-amplitude refocusing phase optimization.
`ExcitationOptimizationResult`	Bounded fixed-amplitude target-excitation or inverse-excitation optimization.
`TunedExcitationInversePipelineResult`	Selected-refocusing to excitation/inverse-excitation pipeline result.
`SLSEResult`	NQR SLSE echo train, local orientation signals, and transition metadata.
`SLSESweepResult`	NQR SLSE selected-echo amplitudes and effective decay estimates across a sweep.
`NQRFIDDistributionResult`	EFG-distribution FID, FFT spectrum, and isochromat metadata.
`SLSEAcquisitionSpectrumResult`	Finite-window SLSE echo acquisition and spectrum, including optional noise and deconvolution metadata.
`WeakB0SpectrumResult`	Static weak-B0 transition spectrum and perturbation-ratio metadata.
`PopulationTransferResult`	NQR perturbation-plus-SLSE signal, reference signal, and normalized difference.
`RelaxationExchange2DResult`	Encode-mix-detect T2-T2 relaxation exchange (REXSY) data and longitudinal mixing propagator.

11.2 High-Level Workflows

```
from spin_dynamics.workflows import (
    run_ideal_cpmg, run_tuned_cpmg, run_untuned_cpmg, run_matched_cpmg,
```

```
    run_ideal_cpmg_train, run_tuned_cpmg_train,
    run_untuned_cpmg_train, run_matched_cpmg_train,
    run_ideal_cpmg_ir_train, run_tuned_cpmg_ir_train,
    run_untuned_cpmg_ir_train, run_matched_cpmg_ir_train,
    run_tuned_q_sweep, run_matched_q_sweep,
    run_tuned_mistuning_sweep, run_matched_mistuning_sweep,
    run_tuned_finite_q_sweep, run_untuned_finite_q_sweep,
    run_matched_finite_q_sweep,
    run_tuned_finite_mistuning_sweep,
    run_untuned_finite_mistuning_sweep,
    run_matched_finite_mistuning_sweep,
    run_matched_z_magnetization_q_sweep,
    run_matched_diffusion_cpmg, run_matched_diffusion_q_sweep,
    run_pgse, run_pgse_moment, run_pgse_walkers, run_pgste_walkers,
    run_dde_walkers, run_ogse_walkers,
    pgse_b_value, gradient_moment_b_value,
    run_ideal_time_varying_cpmg_final,
    run_tuned_time_varying_cpmg_final,
    run_untuned_time_varying_cpmg_final,
    run_matched_time_varying_cpmg_final,
    run_ideal_time_varying_amplitude_sweep,
    run_tuned_time_varying_amplitude_sweep,
    run_untuned_time_varying_amplitude_sweep,
    run_matched_time_varying_amplitude_sweep,
    sinusoidal_field_waveform,
    run_ideal_wurst_inversion,
    run_matched_wurst_inversion,
    run_matched_wurst_cpmg,
    run_radiation_damping_fid,
)
```

The main CPMG runners share `numpts` and `maxoffs`. Finite train runners add `num_echoes`, relaxation constants, rephasing controls, optional `auto_refine_grid`, and optional worker controls. Probe-aware train runners also accept `q_value` and `mistuning_offset` where the underlying probe model supports them.

11.3 Imaging API

```
from spin_dynamics.workflows import (
    ImagingFieldMaps,
    make_imaging_field_maps,
    load_imaging_field_maps_npz,
    run_ideal_phase_encoded_cpmg_imaging,
    run_tuned_phase_encoded_cpmg_imaging,
    run_matched_phase_encoded_cpmg_imaging,
    run_t1_encoded_phase_encoded_cpmg_imaging,
    run_spin_warp_imaging, run_rare_imaging, imaging_slice_sensitivity,
    reconstruct_image_from_kspace,
    form_imaging_image,
    fit_imaging_echo_decay,
    summarize_imaging_noise_trials,
)
```

Prefer the phase_encoded names for new code. The shorter compatibility aliases remain available:

```
run_ideal_cpmg_imaging
run_tuned_cpmg_imaging
run_matched_cpmg_imaging
```

11.4 Parameter Constructors

```
from spin_dynamics.parameters import (
    set_params_ideal,
    set_params_ideal_fid,
    set_params_tuned_orig,
    set_params_untuned_orig,
    set_params_matched_orig,
    set_params_tuned_spa,
    set_params_untuned_spa,
    set_params_matched_spa,
    set_params_tuned_jmr,
    set_params_untuned_jmr,
    set_params_matched_jmr,
)
```

These constructors return dataclass instances mirroring MATLAB sp, pp, and params structures. They accept optional numpts arguments for lightweight tests and examples while preserving MATLAB defaults when omitted.

11.5 Analysis API

```
from spin_dynamics.analysis import (
    t2_kernel, t1_kernel, diffusion_kernel, laplace_kernel,
    invert_laplace_1d, invert_laplace_2d,
    invert_t2, invert_t1, invert_t1_t2, invert_d_t2, invert_t2_t2,
    default_regularization_strengths,
    estimate_noise_rms_from_snr,
    expected_residual_norm_from_snr,
    select_regularization_1d,
    select_regularization_2d,
)
```

For one-dimensional relaxation distributions, use:

```
invert_t2
invert_t1
```

For separable two-dimensional maps, use:

```
invert_t1_t2
invert_d_t2
invert_t2_t2
```

The select_regularization_* helpers choose Tikhonov strengths from an RMS SNR estimate.

11.5.1 Chemical / Site Exchange

The spin_dynamics.exchange namespace provides the Bloch-McConnell site exchange model described in Section 8.14:

```
from spin_dynamics.exchange import (
    ExchangeSite, ExchangeSystem, RelaxationExchange2DResult,
    two_site_exchange, exchange_generator, transverse_generator,
    transverse_propagator, mixing_propagator,
    simulate_exchange_fid, exchange_spectrum,
    simulate_relaxation_exchange_2d,
)
```

11.5.2 Internal / Susceptibility Gradients

The spin_dynamics.susceptibility namespace provides the internal-field generator described in Section 8.15:

```
from spin_dynamics.susceptibility import (
    CylindricalInclusion, SusceptibilityField, InternalGradientDistribution,
    susceptibility_offresonance_map, make_susceptibility_field_maps,
    internal_gradient_maps, internal_gradient_distribution,
)
```

11.6 Optimization Diagnostics

Optimization helpers expose compact NumPy/SciPy-compatible diagnostics for phase-program searches:

```
from spin_dynamics.optimization import run_tuned_refocusing_multistart

result = run_tuned_refocusing_multistart(num_segments=3, num_starts=4)
```

The plotting-free analysis helpers convert MATLAB-style result cells into score summaries, and the tuned excitation/inverse pipeline carries a selected refocusing candidate into bounded target-excitation and inverse-cancellation tests. The inverse stage is still best treated as a parity and diagnostics surface while strong inverse-pulse cancellation remains under validation.

11.7 Core Numerical API

```
from spin_dynamics.core.echo import (
    calc_time_domain_echo,
    calc_time_domain_echo_arb,
    calc_fid_time_domain,
)
from spin_dynamics.core.rotations import (
    calc_rotation_matrix,
    calc_rot_axis_arba3,
    calc_rot_axis_arba4,
    calc_v0crit,
    sim_spin_dynamics_asymp_mag3,
```

```
    sim_spin_dynamics_exc,
)
from spin_dynamics.core.kernels import (
    sim_spin_dynamics_arb7,
    sim_spin_dynamics_arb10,
    sim_spin_dynamics_arb10_chunked,
    sim_spin_dynamics_arb10_diffusion,
    sim_spin_dynamics_arb10_radiation_damping,
)
from spin_dynamics.core.isochromats import (
    analyze_rephasing,
    check_rephasing,
    estimate_rephase_time,
    recommended_numpts_for_rephasing,
)
```

Core functions are intended for validation, debugging, and continued porting. They are closer to MATLAB's array contracts than the workflow layer.

11.8 Probe and Pulse API

```
from spin_dynamics.probes.tuned import (
    tuned_probe_lp,
    tuned_probe_rx,
    tuned_probe_rx_tf,
    calc_rot_axis_tuned_probe_lp_orig2,
    calc_masy_tuned_probe_lp_orig,
)
from spin_dynamics.probes.untuned import (
    untuned_probe_lp,
    untuned_probe_rx,
    untuned_probe_rx_tf,
    calc_rot_axis_untuned_probe_lp,
    calc_masy_untuned_probe_lp,
)
from spin_dynamics.probes.matched import (
    matching_network_design2,
    find_coil_current,
    find_coil_current_wurst,
    matched_probe_rx,
    calc_rot_axis_matched_probe,
    calc_masy_matched_probe_orig,
    calc_masy_matched_nut,
)
from spin_dynamics.pulses import (
    quantize_phase,
    create_wurst_pulse,
    adjust_untuned_segment_lengths,
    tuned_rectangular_pulse_response,
    untuned_rectangular_pulse_response,
    matched_rectangular_pulse_response,
    matched_wurst_pulse_response,
)
```

11.9 Noise, Motion, and Radiation Damping

```
from spin_dynamics.noise import (
    NoiseSpec,
    NoiseMetadata,
    MatchedFilterSNRResult,
    add_received_noise,
    estimate_matched_filter_snr,
    ideal_noise_density,
    tuned_probe_output_noise_density,
    untuned_probe_output_noise_density,
    matched_probe_output_noise_density,
)
from spin_dynamics.motion import (
    ParticleEnsemble,
    MotionFieldMaps2D,
    make_motion_field_maps_2d,
    initialize_ensemble_from_density,
    make_semipermeable_plane,
    advect_diffuse_positions,
    move_ensemble,
    apply_free_precession,
    apply_rf_rotation,
    free_precession_with_motion_step,
    receive_signal,
    transverse_b1_magnitude,
)
from spin_dynamics.sequences.motion import (
    MotionSequenceStep,
    MotionSequenceResult,
    make_motion_cpmg_sequence,
    make_motion_udd_sequence,
    run_motion_sequence,
    run_motion_cpmg_sequence,
    run_motion_udd_sequence,
)
from spin_dynamics.sequences import (
    cpmg_pulse_times,
    udd_pulse_times,
    interval_durations,
    toggling_frame_integral,
    dephasing_filter_function,
)
from spin_dynamics.radiation_damping import (
    radiation_damping_time,
    proton_thermal_magnetization_density,
    water_proton_sample,
    hyperpolarized_proton_sample,
    normalized_radiation_damping_weights,
    radiation_damping_probe_from_tuned,
    radiation_damping_probe_from_matched,
    initial_state_from_flip_angle,
    analytic_radiation_damping_envelope,
    simulate_radiation_damping,
```

```
    simulate_radiation_damping_fid,
    simulate_nmr_maser,
)
```

11.10 Scalar Coupling API

```
from spin_dynamics.coupling import (
    CoupledSpinSystem,
    CoupledIsochromatEnsemble,
    CoupledIsochromatStep,
    CoupledIsochromatSequenceResult,
    coupled_spin_system,
    coupled_isochromat_ensemble,
    free_precession_step,
    rf_step,
    simulate_coupled_isochromat_sequence,
    spin_operator,
    total_operator,
    product_operator,
    zeeman_hamiltonian,
    secular_j_hamiltonian,
    isotropic_j_hamiltonian,
    rf_hamiltonian,
    equilibrium_density,
    evolve_density,
    expectation,
    j_modulation_curve,
    proton_detected_j_modulation,
    carbon_detected_j_modulation,
    tango_b_filter,
    fit_known_j_spectrum,
    JEditingFitResult,
    simulate_slic_spectrum,
    two_spin_slic_transfer_time,
    SLICSpectrumResult,
)
```

Object	Purpose
`CoupledSpinSystem`	Validated small spin-1/2 system with offsets and scalar couplings in hertz.
`CoupledIsochromatEnsemble`	Ensemble of local B0/B1 field samples for one coupled-spin system.
`free_precession_step`, `rf_step`	Sequence-step builders with optional time-varying B0/B1 overrides.
`simulate_coupled_isochromat_sequence`	Dense coupled-spin sequence propagation over an isochromat ensemble.
`spin_operator`, `total_operator`, `product_operator`	Dense spin-1/2 operator builders.
`zeeman_hamiltonian`	Rotating-frame offset Hamiltonian in radians per second.

`secular_j_hamiltonian`	Weak-coupling $I_z I_z$ scalar Hamiltonian.
`isotropic_j_hamiltonian`	Isotropic scalar Hamiltonian for strongly coupled spin-1/2 systems.
`rf_hamiltonian`	RF nutation Hamiltonian for selected spins.
`equilibrium_density`, `evolve_density`, `expectation`	Minimal dense density-matrix utilities.
`j_modulation_curve`	Sum of cosine J-modulation components.
`proton_detected_j_modulation`	Proton-detected low-field J-editing response.
`carbon_detected_j_modulation`	Carbon-detected $\cos^n$ J-editing response for CH_n groups.
`tango_b_filter`	Ideal TANGO-B scalar-coupling filter envelope.
`fit_known_j_spectrum`	Least-squares amplitudes for supplied J-coupling positions.
`simulate_slic_spectrum`	Spin-lock response scan for dense coupled-spin systems.

11.11 ESR API

```
from spin_dynamics.esr import (
    ESRSpinSystem,
    ESRRelaxationModel,
    ESRDistributionSample,
    NuclearSite,
    ElectronNuclearSystem,
    resonance_frequency_hz,
    resonance_field_tesla,
    effective_g_value,
    simulate_field_sweep,
    simulate_frequency_spectrum,
    static_disorder_grid,
    simulate_field_sweep_distribution,
    simulate_frequency_spectrum_distribution,
    gaussian_lineshape,
    lorentzian_lineshape,
    flip_angle_duration,
    simulate_fid,
    simulate_hahn_echo,
    electron_nuclear_system,
    diagonalize_hyperfine_system,
    simulate_hyperfine_field_sweep,
)
```

Object	Purpose
`ESRSpinSystem`	Single-electron spin-1/2 ESR system with scalar, diagonal, or full g tensor.
`resonance_frequency_hz`, `resonance_field_tesla`	Convert between field and microwave frequency for one ESR orientation.
`effective_g_value`	Direction-dependent $g_{\mathrm{eff}} = \lvert\mathbf{g}^{\mathsf{T}}\hat{\mathbf{n}}\rvert$.

`simulate_field_sweep`, `simulate_frequency_spectrum`	Single-crystal or powder CW ESR spectra with Gaussian/Lorentzian absorption or derivative display.
`gaussian_lineshape`, `lorentzian_lineshape`	Unit-height absorption profiles and first derivatives.
`static_disorder_grid`	Weighted diagonal g-strain and applied-field-offset samples.
`simulate_field_sweep_distribution`	Field-swept ESR spectrum averaged over static disorder samples.
`flip_angle_duration`	Rectangular-pulse duration for a spin-1/2 flip angle and nutation rate.
`simulate_fid`, `simulate_hahn_echo`	Rotating-frame pulsed ESR FID and Hahn-echo density-matrix simulations.
`ESRRelaxationModel`	Phenomenological Liouville-space T_1/T_2 relaxation model for pulsed ESR.
`NuclearSite`, `ElectronNuclearSystem`	Dense electron-nuclear product-space hyperfine model containers.
`electron_nuclear_system`	Convenience builder for isotropic electron-nuclear hyperfine systems.
`diagonalize_hyperfine_system`	Exact dense energy levels and ESR-active transitions for a hyperfine system.
`simulate_hyperfine_field_sweep`	Field-swept ESR spectrum including isotropic electron-nuclear hyperfine coupling.

11.12 NQR API

```
from spin_dynamics.nqr import (
    QuadrupolarSite,
    NQRTransition,
    NQREigensystem,
    spin_matrices,
    quadrupole_frequency_scale_hz,
    quadrupole_hamiltonian,
    zeeman_hamiltonian,
    nqr_hamiltonian,
    diagonalize_site,
    OrientationSample,
    single_crystal_orientation,
    powder_average_grid,
    b0_b1_powder_average_grid,
    b0_powder_average_grid,
    SelectivePulse,
    apply_selective_pulse,
    transition_drive_scale,
    NQRRelaxationModel,
    slse_sequence,
    simulate_slse,
    simulate_slse_offset_sweep,
    simulate_slse_spacing_sweep,
    gaussian_efg_distribution,
    temperature_efg_distribution,
```

```
    simulate_fid_efg_distribution,
    simulate_slse_acquisition_spectrum,
    simulate_weak_b0_spectrum,
    weak_field_ratio,
    zeeman_frequency_hz,
    simulate_population_transfer,
    SLSEResult,
    PopulationTransferResult,
    WeakB0SpectrumResult,
)
```

Object	Purpose
`QuadrupolarSite`	One quadrupolar nucleus in the EFG principal-axis system.
`spin_matrices`	Dense angular-momentum operators for arbitrary integer or half-integer spin.
`quadrupole_frequency_scale_hz`	Spin-dependent Hamiltonian scale for spin-1 and spin-3/2 frequency conventions.
`quadrupole_hamiltonian`	Zero-field quadrupole Hamiltonian in radians per second.
`zeeman_hamiltonian`	Optional weak-B0 perturbation in radians per second.
`diagonalize_site`	Energy levels, transition frequencies, and transition dipole vectors.
`OrientationSample`	One EFG orientation relative to lab RF and optional B0 fields.
`single_crystal_orientation`	One fixed orientation sample.
`powder_average_grid`	Normalized spherical powder quadrature grid.
`b0_b1_powder_average_grid`	Powder grid with correlated static-field and RF-field directions.
`SelectivePulse`	Rectangular selective pulse on one transition.
`apply_selective_pulse`	Embedded two-level RF propagation in the energy basis.
`NQRRelaxationModel`	Phenomenological Liouville-space population/coherence relaxation rates.
`slse_sequence`	Convenience builder for SLSE detection.
`simulate_slse`	Selective-pulse SLSE echo train for a fixed orientation or powder.
`simulate_slse_offset_sweep`, `simulate_slse_spacing_sweep`	SLSE diagnostics versus RF offset or pulse period.
`gaussian_efg_distribution`, `temperature_efg_distribution`	Static EFG isochromat distributions.
`simulate_fid_efg_distribution`	Time-domain FID and FFT spectrum from an EFG distribution.
`simulate_slse_acquisition_spectrum`	Finite-window acquired SLSE echo and spectrum, with noise and optional deconvolution.
`simulate_weak_b0_spectrum`	Static weak-B0 transition spectrum for spin-1 or spin-3/2.
`weak_field_ratio`, `zeeman_frequency_hz`	Weak-field regime checks based on $\lvert\gamma B_0\rvert/\nu_{\mathrm{ref}}$.

`simulate_population_transfer`	Perturbation pulse plus SLSE detection for 2D NQR diagnostics.

11.13 Optimization API

```
from spin_dynamics.optimization import (
    spa_pulse_list,
    rectangular_refocusing_lengths,
    evaluate_tuned_refocusing_pulse,
    evaluate_untuned_refocusing_pulse,
    evaluate_matched_refocusing_pulse,
    evaluate_ideal_v0crit_refocusing_pulse,
    evaluate_ideal_v0crit_excited_refocusing_pulse,
    evaluate_ideal_time_varying_refocusing_pulse,
    optimize_tuned_refocusing_phases,
    optimize_untuned_refocusing_phases,
    optimize_matched_refocusing_phases,
    optimize_ideal_v0crit_refocusing_phases,
    optimize_ideal_v0crit_excited_refocusing_phases,
    optimize_ideal_time_varying_refocusing_phases,
    run_tuned_refocusing_multistart,
    run_untuned_refocusing_multistart,
    run_matched_refocusing_multistart,
    run_ideal_v0crit_refocusing_multistart,
    run_ideal_v0crit_excited_refocusing_multistart,
    run_ideal_time_varying_refocusing_multistart,
    evaluate_tuned_excitation_pulse,
    evaluate_tuned_inverse_excitation_pulse,
    optimize_tuned_excitation_phases,
    optimize_tuned_inverse_excitation_phases,
    run_tuned_excitation_multistart,
    run_tuned_inverse_excitation_multistart,
    run_tuned_excitation_inverse_pipeline,
    multistart_to_matlab_results,
    save_multistart_results_npz,
    load_optimization_results,
    summarize_matlab_results,
    select_matlab_result_program,
    analyze_matlab_optimization_results,
    analyze_tuned_inverse_result_pair,
)
```

The optimization layer is useful for compact pulse-design studies and MATLAB-style result inspection. Broad historical `fmincon` parity and strong inverse-excitation cancellation parity remain validation targets.

12 Known Gaps

The Python port is intentionally conservative where MATLAB parity still matters. The largest remaining gaps are broader high-Q matched-probe validation, full historical MATLAB `.mat` structure parity, broad `fmincon` parity, paper-level scalar-coupling fixture comparisons, relaxation-aware coupled-spin pulse-sequence runners, full nonselective quadrupolar pulse propagation, experimental NQR fixture comparisons, ESR anisotropic hyperfine and multi-electron interactions, and acceleration backends beyond the current NumPy implementation.

New work should add small reference fixtures first, then a NumPy implementation, then optional acceleration or plotting once numerical behavior is pinned down.